VIA University College

Software Technology Engineering.

1st Semester

Cloverville

Project Report

3 Students:

Diana Boiko (360245)

Marcus-David Muresan (361734)

Maciej Religa (363666)

Supervisors:

Line Lindhardt Egsgaard

Michael Viuff

Hand-in Date:

19.12.2025

Characters:

42233

Table of Content

1. Abstract **3**
2. Introduction **4**
3. Main Section **5**
3.1. Analysis 7
3.1.1. Summary 7
3.1.2. Requirements 7
3.1.3. Functional Requirements 7
3.1.4. Non-Functional Requirements 9
3.1.5. Use Case Diagram 9
3.1.6. Use Case Descriptions 10
3.1.7. Relation Between Requirements and Use Cases 13
3.1.8. Activity Diagrams 15
3.1.9. Domain Model 17
3.2. Design 18
3.2.1. Class Diagram 18
3.2.2. Sequence Diagrams 19
3.2.3. Custom App Design 21
3.2.4. Custom Website Design 21
3.3. Implementation 22
3.3.1. View 22
3.3.2. Scene Management 22
3.3.3. Model and Application Logic 22
3.3.4. Data Structures and Lists 23
3.3.5. Task Completion and Participation Boost 23
3.3.6. File Persistence (JSON Export & Import) 23
3.3.7. Website Integration 25
3.3.8 User Guide and Usability Support 25
3.3.9. Summary 26
3.4. Testing 26
4. Results and Discussion **27**
4.1. Results 27
4.2. Discussion 29
5. Conclusion and Recommendations **30**
6. References **31**
7. Appendices **32**

1. Abstract

This report presents a software solution developed for Cloverville, an ecological village that faced issues with its manual administration. Previously, the community relied on a physical notebook kept by the coordinator, Green Bob. This method led to calculation errors and a lack of transparency regarding resident contributions. The main goal of this project was to replace the paper system with a digital tool to ensure fairness and allow residents to track their green points easily.

The solution consists of two main parts. The first is a desktop application built with Java and JavaFX, which serves as the control center for the administrator. It allows for the management of resident profiles, communal tasks, and trade offers. The second part is a website built with HTML, CSS, and JavaScript, designed to show the community's progress.

We chose to connect these two components using a JSON file instead of a complex database. This decision keeps the system lightweight and easy to maintain, fitting the village's needs. The Java app saves data to the JSON file, and the website reads it to display current information. To design the system, we used standard UML diagrams, such as Domain Models and Activity Diagrams, to structure the code logic.

The project resulted in a functional software prototype that replaces the previously manual notebook-based system with a centralized digital solution. The implemented system fulfills the defined functional requirements by enabling the registration, visibility, and management of residents' contributions, while the non-functional requirements regarding technology choice and anonymity were also met. Additionally, the website uses responsive design, making it accessible on both desktop and mobile devices. The project proves that a simple IT system can effectively improve the management of a small community.

2. Introduction

Cloverville is an ecological village built on principles of sustainability, shared responsibility, and community cooperation (Limbu, 2024). Residents contribute to the community through green actions, communal tasks, and local trade offers, all of which support both individual engagement and collective progress. However, as described in the project case, these contributions are currently recorded manually in a physical notebook maintained by the village coordinator, Green Bob. This manual approach has proven to be inefficient, error-prone, and difficult to maintain, resulting in limited transparency, inconsistent point calculation, and reduced motivation among residents. During the project description phase, the following problem statement was made to capture the core challenge of the case:

How can Cloverville establish a centralized digital system for recording contributions and tracking points that improves data accessibility and accuracy, while simultaneously fostering motivation and aligning with the community's cooperative values.

Despite Cloverville's strong commitment to sustainable living, the community currently relies on a manual, notebook-based system to record green actions, communal tasks, and trade offers. This approach creates an administrative bottleneck, limits data accessibility, and reduces transparency, as all information is stored in a single physical location and managed by one individual. As a result, residents cannot easily track their own contributions or compare efforts over time, which may lead to perceptions of unfairness and reduced motivation. The lack of a centralized digital system therefore represents both a practical and social challenge for the community, directly affecting all residents who participate in shared activities.

Many existing task management or marketplace systems emphasize competition, profit, or continuous online connectivity, which conflicts with Cloverville's cooperative and low-tech values. To address this gap, the project proposes a lightweight digital solution consisting of a JavaFX desktop application for administrative tasks and a public website for information sharing. The system is designed to be simple, transparent, and easy to use, while maintaining anonymity where required and minimizing technical complexity.

From an academic perspective, the project applies principles from software engineering and problem-based learning. Standard UML techniques, including use case diagrams, activity diagrams, and domain models, are used to analyse the problem domain and design a structured solution. The project integrates theoretical knowledge from courses such as Software Development with Java (PRO) and Web Development (WEB) into a coherent, practical system.

The project is significant in both real-world and academic contexts. Socially, it supports fairness, motivation, and visibility of contributions within the community (Deci & Ryan, 2000). Environmentally, it encourages sustainable behaviour while using a low-energy, lightweight technical solution without reliance on databases or continuous internet connectivity. Technologically, it demonstrates how well-structured but simple IT systems can effectively support small communities without unnecessary complexity.

3. Main Section

Methods

To develop the Cloverville system, a combination of analytical, design-oriented, and implementation-focused methods was applied. These methods were selected to ensure that the solution was both technically sound and aligned with the values and constraints described in the project case.

Methods for Gathering Knowledge

The primary source of knowledge for the project was the Cloverville case description, which includes an interview with the client, Green Bob. This document was analysed in detail to understand the problem domain, identify system actors, and derive functional and non-functional requirements. In addition, previous SEP1 projects were reviewed to gain insight into expected documentation structure and system scope.

Throughout the project, discussions with the supervisor supported the interpretation of requirements and helped clarify design decisions, particularly during the analysis and modelling phases.

Methods for Analysis, Design, and Testing

Analysis and Design Methods

The analysis phase followed a structured and chronological approach. First, functional and non-functional requirements were defined. Based on these, a use case diagram was created to identify the main system functionalities and actors. Detailed use case descriptions were then written to specify system behaviour, including normal flows and alternative error scenarios.

To further describe system logic, activity diagrams were created for selected use cases. A domain model was developed to represent the core concepts of the problem domain and their relationships. This model later informed the class diagram and sequence diagrams, which guided the implementation in Java.

All UML diagrams were created using Astah, ensuring consistency between analysis, design, and code structure.

Prototyping

Before implementation, a visual prototype of the website was created using Figma. This helped define the layout, navigation structure, and overall visual style inspired by Cloverville's ecological theme. No prototype was created for the JavaFX application; instead, the interface evolved iteratively during development using SceneBuilder and FXML.

Testing Methods

Testing was conducted manually on the developers laptops. The JavaFX application was tested by going through the defined use cases and verifying that each requirement was fulfilled. Particular attention was given to CRUD operations, point calculations, and error handling.

The integration between the JavaFX application and the website was also tested by updating data in the admin application, exporting JSON files, and verifying that the website displayed the updated

information correctly. These tests ensured consistency between system components and confirmed correct behaviour according to the requirements.

Project Activities

The project activities were carried out chronologically, following the phases of analysis, design, and implementation.

During the initial phase, the group focused on analysis and modelling. Requirements were defined, and UML diagrams were created to establish a clear system structure. At the same time, project organisation tools were set up, including Trello for task management and Discord for communication and resource sharing.

Once the design phase was completed, implementation began during the three-week project period. GitHub repository was created for the website to support collaboration (see Appendix L). Development was mainly done through in-person meetings, as the group found it easier to focus and discuss structure when working together offline.

Tasks were divided among group members based on system components and features. The JavaFX application was implemented in parallel with the website, while regular discussions ensured that JSON structures and data flow remained consistent. Feedback was also exchanged with other student groups to validate progress and clarify supervisor expectations.

Results

The project resulted in two main deliverables: a JavaFX-based administrator application and a responsive website.

The JavaFX application allows the administrator to manage residents, communal tasks, trade offers, and point systems. It supports registering communal participation, recording green actions, applying participation boosts, performing periodic point resets, and exporting data to JSON files.

The website dynamically displays trade offers, weekly green focus tasks, community green points, and an anonymous catalogue of green actions. All displayed information is loaded from JSON files generated by the admin application, ensuring that the website always reflects the latest system state.

Manual testing confirmed that the implemented functionalities meet the defined requirements and that data updates are correctly reflected across both system components.

Ethical Considerations

Several ethical aspects were considered during the project. From an environmental perspective, the system was designed as a lightweight solution using JSON files instead of a database, reducing resource usage and system complexity.

From a social perspective, resident privacy was respected by displaying green actions anonymously on the website, preventing unwanted comparison or competition. Accessibility was also considered in the website design through readable layouts and clear navigation.

3.1. Analysis

3.1.1. Summary

This chapter presents the analysis of the Cloverville project. The purpose of this phase was to translate the client's need for a digital administration tool into clear technical specifications. We began by defining the Functional and Non-Functional Requirements to establish what the system must do. These requirements were further explored using Use Case Diagrams to visualize the interactions between Green Bob and the application. Additionally, we created Activity Diagrams to map out key processes and a Domain Model to structure the system's data. This analysis serves as the blueprint for the Design and Implementation chapters.

3.1.2. Requirements

To understand the needs of the Cloverville community and define what the system must achieve, we identified a set of user-centred requirements based on the client interview and project objectives. The requirements are organized in a prioritized list ranging from 1 to 19. These requirements describe both what the system should do (functional requirements) and how it should behave (non-functional requirements). They are written as user stories to clearly express the purpose of each feature and are numbered to reflect their priority and logical order within the system(see Appendix B).

3.1.3. Functional Requirements

01 – Create Resident Records

As an Admin, I want to create new resident records so that I can register their activities in the system.

02 – Manage Resident Information

As an Admin, I want to view and update resident information so that I can keep member details accurate and up to date.

03 – Remove Residents

As an Admin, I want to remove residents so that only active members can be selected for new activities.

04 – Manage Communal Tasks

As an Admin, I want to create and view communal tasks so that their descriptions are clear for all residents.

05 – Create Trade Offers

As an Admin, I want to add new trade offers so that residents can exchange goods and services through the system.

06 – Update Trade Offers

As an Admin, I want to update existing trade offers so that information and availability stay accurate.

07 – Remove Trade Offers

As an Admin, I want to delete or remove outdated trade offers so that only valid and available offers are shown.

08 – Register Communal Participation

As an Admin, I want to register resident's participation in a communal task so that they receive personal points.

09 – Register Green Actions Anonymously

As an Admin, I want to add green actions done by residents to the system and display them anonymously on the website so that others can get inspired without comparison.

10 – Assigning Community Points

As an Admin, I want the system to automatically add community points when a green action is recorded, based on the configured rules, so that the community total updates correctly.

11 – Publish Website Data

As an Admin, I want to publish updated data (trade offers, green actions, points) to JSON file so that residents and visitors always see the most up-to-date information on the website.

12 – Define Community Point Rules

As an Admin, I want to define and adjust how many community points each type of green action gives so that the system can calculate points consistently and fairly.

13 – View Trade Offers

As a Resident, I want to see the current trade offers on the website so that I can find opportunities to trade goods and services.

14 – View Weekly Green Focus Tasks

As a Resident, I want to see the week's green focus tasks so that I know which activities the community is focusing on.

15 – View Community Green Points

As a Resident, I want to see the total community green points and the progress toward the next milestone so that we know how close we are to unlocking the next community event.

16 – Browse Green Actions

As a Website Visitor, I want to browse the green actions taking place in Cloverville so that I can understand the community and get inspired.

17 – Configure Participation Boost

As an Admin, I want to configure a participation boost for residents who have not joined any communal tasks within a defined time period so that participation is encouraged fairly.

18 – Auto-Transfer Unused Personal Points

As an Admin, I want unused personal points to be automatically transferred to the community pool during a scheduled reset so that they continue to support the village's shared goals.

19 – Periodic Personal-Point Reset

As an Admin, I want to run a periodic reset of personal points (monthly/quarterly) so that no one becomes a "point king".

3.1.4. Non-Functional Requirements

NFR1 — Technology Implementation

The Cloverville administration system must be implemented in Java using JavaFX, and all exported data must be displayed clearly on the public HTML/CSS/JavaScript website through JSON files.

NFR2 — Anonymity

The system must ensure that all green actions published on the website are anonymized by excluding resident names, identifiers, and contact details.

3.1.5. Use Case Diagram

The use case diagram provides a high-level overview of the Cloverville system and shows how the different actors interact with its main functionalities. The Admin is the primary actor and is responsible for all internal management tasks, including residents, communal tasks, trade offers, points, and website data. To keep the diagram readable, several related actions are grouped into higher-level use cases such as Manage Residents, Manage Communal Tasks, and Manage Trade Offers. These grouped use cases include add, edit, and delete operations, which are detailed further in the use case descriptions.(see Appendix C)

A key aspect of the diagram is the clear separation between administrative functionality and public access. While the Admin has access to all management and configuration features, Website Visitors can only interact with the system through the View Website Content use case.

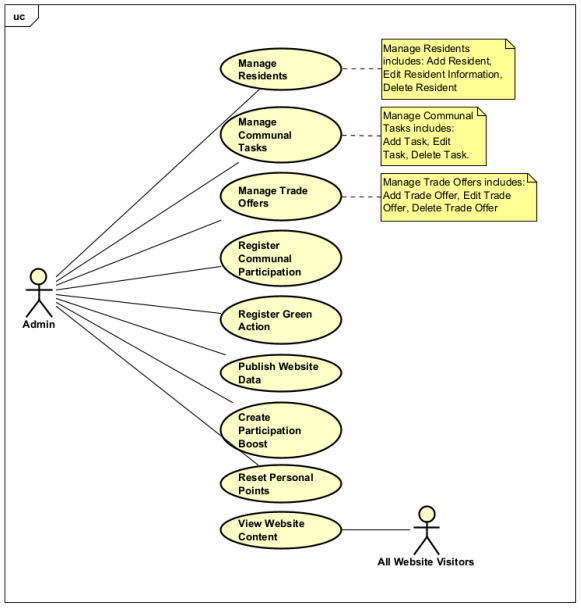

Figure 1. Use Case Diagram

3.1.6. Use Case Descriptions

Manage Residents Use Case

The Manage Residents use case represents one of the core functionalities of the system. This use case groups the following actions: Add Resident, Edit Resident Information, and Delete Resident. These actions allow the administrator to maintain accurate and up-to-date resident data, which is required for participation tracking, point assignment, and task registration. (see Appendix D)

Add Resident Use Case

This functionality allows the administrator to create a new resident by entering the required information. Once confirmed, the resident becomes available for selection in communal tasks and green actions. Through the relationship table between use cases and requirements, this use case covers requirement 01.

An alternate sequence is present when required fields are missing or when a duplicate resident already exists. These cases are further illustrated in the corresponding activity diagram.

1. **Manage Residents**

Use case

Manage Residents: Add residents.

Summary

Create a new resident in the system.

Actor

Administrator

Precondition

The application is running.

Postcondition

A new resident record is created and visible in the resident list.

Base Sequence

1. Administrator opens the Residents menu.
2. System displays the list of residents.
3. Administrator selects “Add Resident”.
4. System shows an empty resident form.
5. Administrator fills in the required information.
6. Administrator confirms the creation.
7. System shows a confirmation message, and the new resident appears in the list.

Alternate Sequence

[ALT1] (Step 5): If required fields are left empty, the system shows an error message and the administrator stays on the form.

[ALT2] (Step 6): If the administrator cancels, the system returns to the resident list without creating a new resident.

[ALT3] (Step 6/7): If a resident with the same information already exists, the system shows a duplicate warning and returns to the form.

Note

Covers requirements 01.

Figure 2. Use case: Manage Residents: Add Residents

Edit Resident Information Use Case

This functionality enables the administrator to update existing resident information by selecting a resident and modifying one or more fields. The changes are applied only after confirmation. Through the relationship table, this use case covers requirement 02.

An alternate sequence is triggered if required fields are left empty or if the administrator cancels the action.

Use case

Manage Residents: Edit Resident Information

Summary

Update existing resident data.

Actor

Administrator

Precondition

At least one resident exists in the system.

Postcondition

Chosen resident's information is updated and shown with new values.

Base Sequence

1. Administrator opens the Residents menu.
2. System displays the list of residents.
3. Administrator selects a resident to edit.
4. System shows the current details for the selected resident.
5. Administrator changes one or more fields.
6. Administrator confirms the changes.
7. System shows a confirmation message and the updated resident details.

Alternate Sequence

[ALT2] (Step 5): If the administrator leaves required field empty, the system shows an error and stays on the edit form.

[ALT3] (Step 6): If the administrator cancels, the system returns to the resident list without saving changes.

Note

Covers requirements 02.

Figure 3. Use case: Manage Residents: Edit Residents

Delete Resident Use Case

This functionality allows the administrator to remove a resident from the system. The action requires confirmation to prevent accidental deletion. Through the relationship table, this use case covers requirement 03.

An alternate sequence occurs if the administrator cancels the deletion.

These three use cases together demonstrate how the “Manage” functionality operates in the system. All other use case descriptions follow the same structure and are included in the Appendix.

Use case

Manage Residents: Delete Resident.

Summary

Remove a resident from the system.

Actor

Administrator

Precondition

At least one resident exists in the system.

Postcondition

The selected resident is removed and no longer appears in the resident list.

Base Sequence

1. Administrator opens the Residents menu.
2. System displays the list of residents.
3. Administrator selects a resident to delete.
4. System shows the resident's details and asks for confirmation.
5. Administrator confirms the removal.
6. System shows a confirmation message and the resident no longer appears in the list.

Alternate Sequence

[ALT2] (Step 5): If the administrator cancels the removal, the system returns to the resident list and no changes are made.

Note

Covers requirements 03.

Figure 4. Use case: Manage Residents: Delete Residents

3.1.7. Relation Between Requirements and Use Cases

Each use case description is linked to the relevant requirements through a relationship table, ensuring traceability between requirements and system functionality (see Appendix E).

Use Case	**Requirement Covered**
UC1 Manage Resident: Add Resident	01
UC1 Manage Resident: Edit Resident Information	02
UC1 Manage Resident: Delete Resident	03
UC2 Manage Communal Tasks: Add Communal Task	04
UC2 Manage Communal Tasks: Edit Communal Task	04
UC2 Manage Communal Tasks: Remove Communal Task	04
UC3 Manage Trade Offers: Add Trade Offer	05

UC3 Manage Trade Offers: Edit Trade Offe	06
UC3 Manage Trade Offers: Delete Trade Offer	07
UC4 Register Communal Participation	08
UC5 Register Green Action	09, 10, 12
UC6 Publish Website Data	11
UC7 Create Participation Boost	17
UC8 Reset Personal Points	18, 19
UC9 View Website Content	13, 14, 15, 16

3.1.8. Activity Diagrams

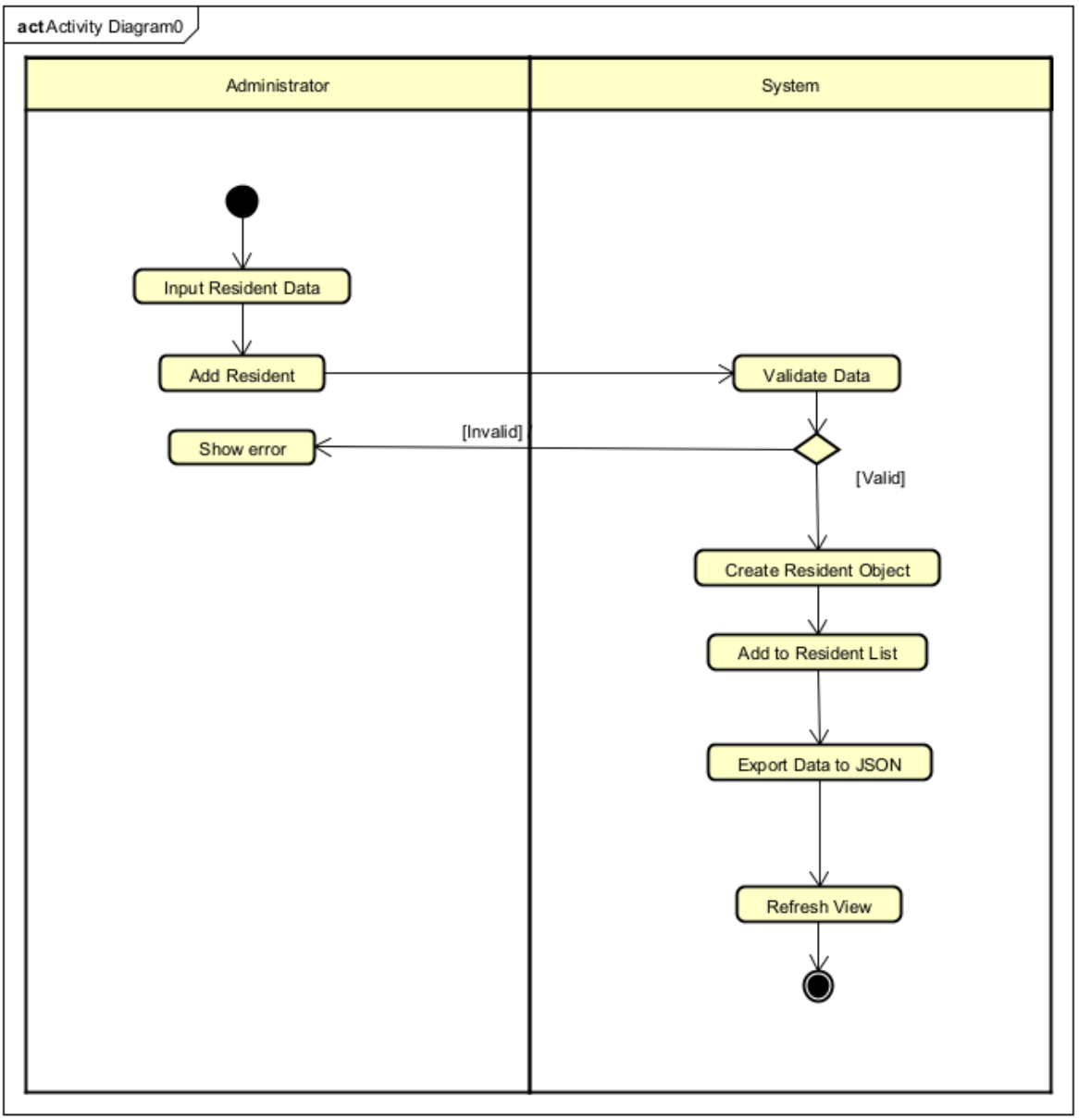

Figure 5. Activity Diagram. Add Residents

The first diagram models the Add Resident workflow, illustrating the interaction between the Administrator and the System partition. This diagram focuses heavily on the system's validation logic, which is a key non-functional requirement for data integrity. The flow demonstrates that before a Resident object is instantiated, the system performs a rigorous check to ensure no required fields are missing and that the resident does not already exist in the database. The decision node clearly splits the flow into a "success path," leading to data persistence, and an "error path," which loops back to prompt the administrator for correction. This modeling ensures that the application prevents invalid states from occurring at the data entry level (see Appendix F).

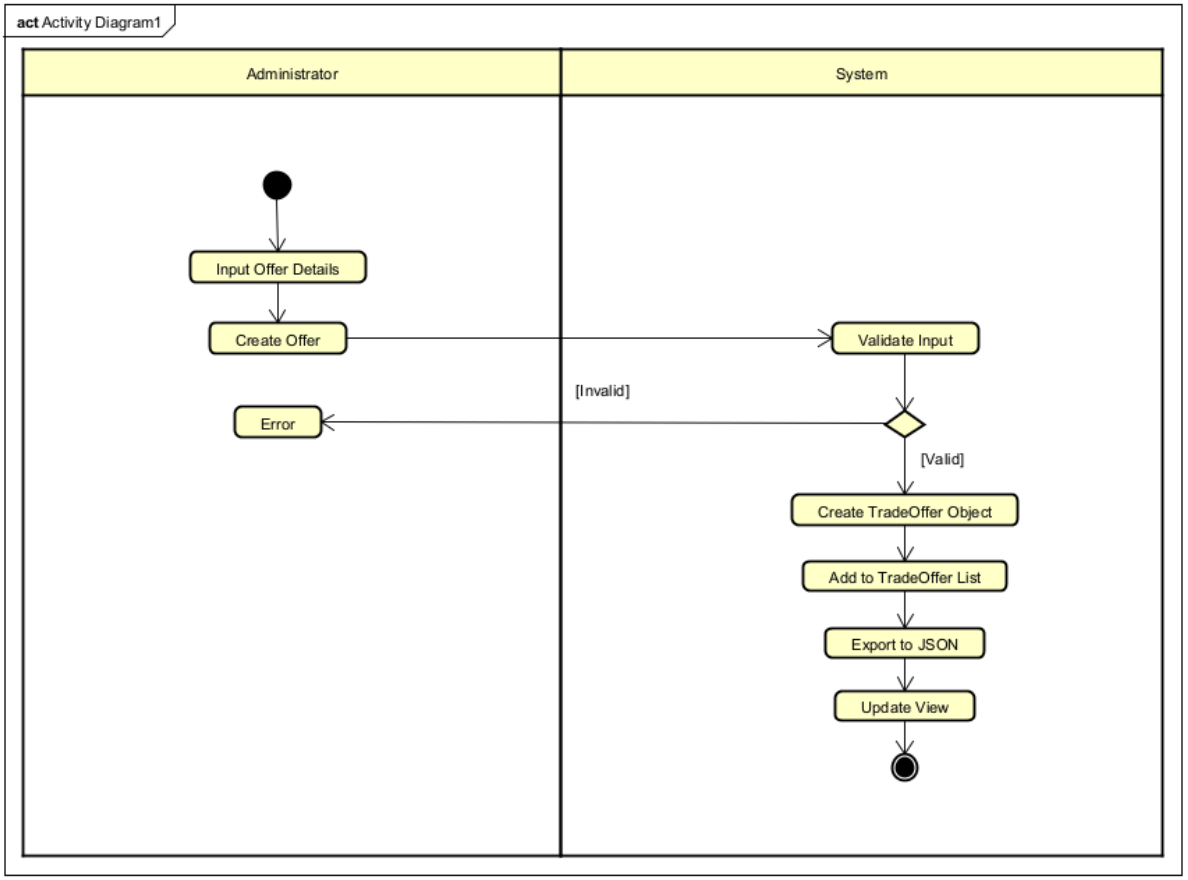

Figure 6. Activity Diagram. Add Trade Offers

The second diagram depicts the Add Trade Offer process. This workflow was selected for detailed modeling because it bridges the gap between the internal Java administration tool and the external web interface. The diagram visualizes the complete lifecycle of a trade offer creation: from the administrator's input, through the validation and object creation, to the final critical step of JSON serialization. As illustrated in the system partition, once the offer is added to the internal list, the system automatically triggers an export function. This ensures that the public website is immediately updated with the new content, satisfying the requirement for real-time information display without manual file handling.

3.1.9. Domain Model

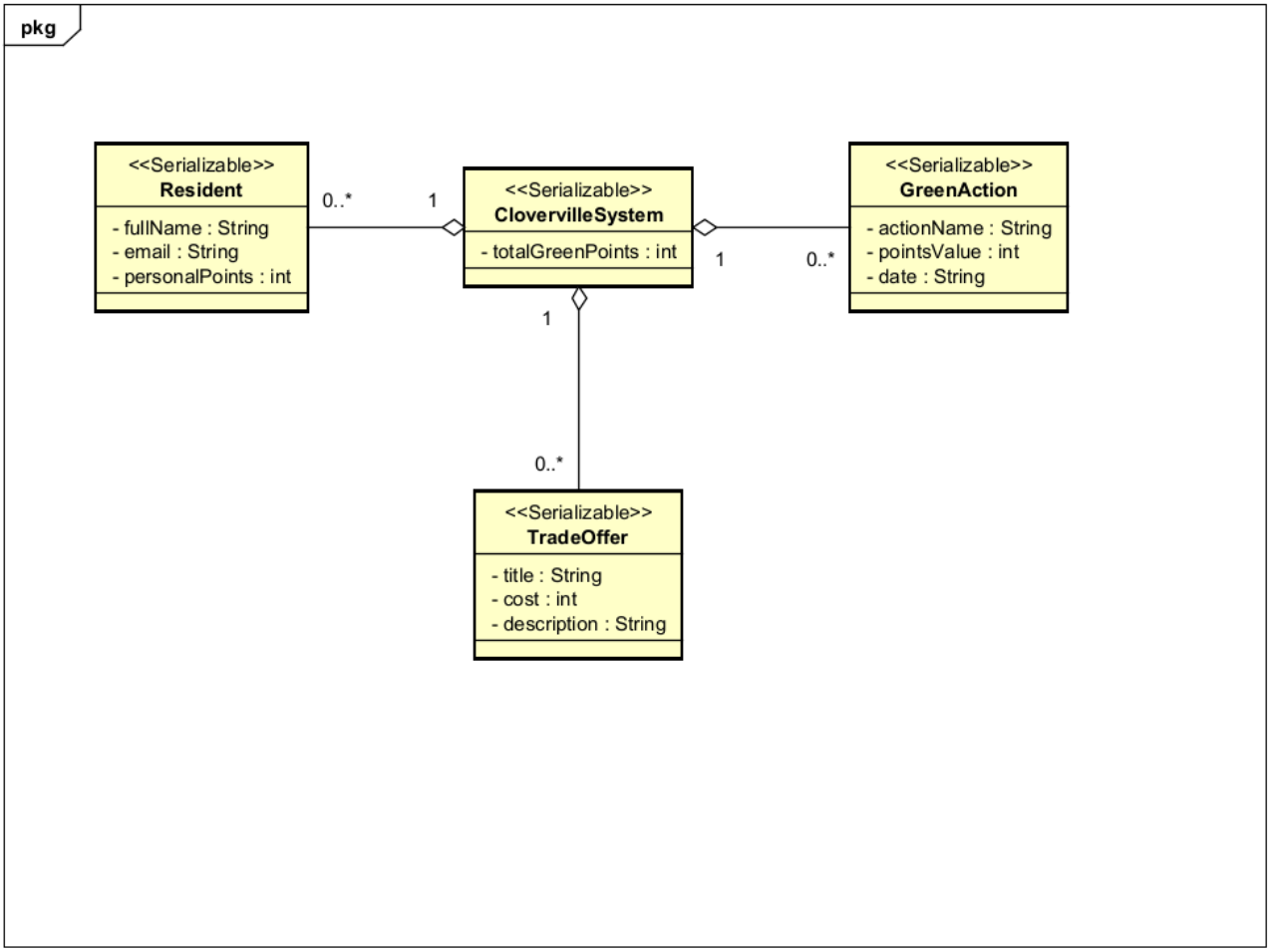

Figure 7. Domain Model

The Domain Model represents the data structure of the application. The system revolves around the Cloverville System class, which holds lists of Residents, Green Actions, and Trade Offers. This design was chosen to organize the data efficiently and to make it easy to save the entire system state to a single file (see Appendix G).

3.2. Design

3.2.1. Class Diagram

The class diagram represents the overall structure of the Cloverville application and illustrates how the system is organised into logical layers. The design follows a clear separation of concerns, where user interface logic, and domain logic are kept distinct. At the centre of the system is the Cloverville Facade, which coordinates all operations and acts as the single access point for controllers(see Appendix H).

The application uses the Facade design pattern. All controllers interact exclusively with the Cloverville Facade rather than directly accessing domain or list classes. This simplifies controller logic, reduces coupling, and centrilises business rules such as point calculations, task completion and data export.

The system is divided into several conceptual layers. The user interface layer consist of JavaFX controllers such as ListController, TradeOffersController, and CompleteTaskController. These controllers handle user interaction and delegate all logic to the Facade.
Below this lies the domain layer, which includes core entities such as Resident, TradeOffer, CommunalTask and GreenAction These classes model the real-word concepts of the Cloverville village.

Each domain class is managed by a corresponding list class, such as ResidentList, TradeOfferList, and CommunalTaskList. These list classes are owned by the Cloverville Facade using composition, meaning their lifecycle is fully controlled by the Facade. This ensures consistent state management and prevents direct manipulation of data from outside the Facade.

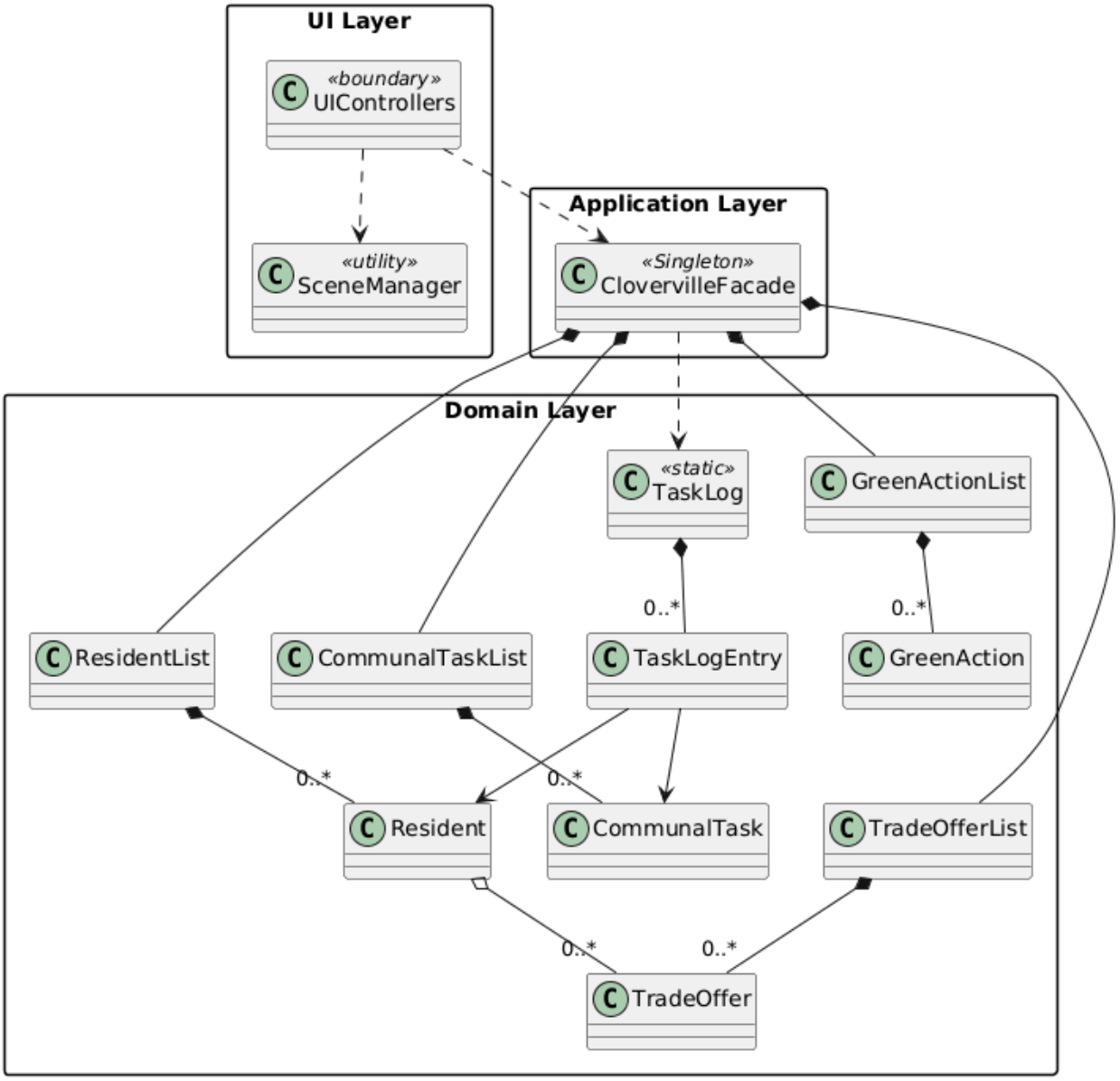

Figure 8. Class Diagram

3.2.2. Sequence Diagrams

The sequence diagram is used as a documentation tool to visualise an already implemented sequence of logic within the Cloverville application. Rather than serving as a planning artefact, the diagram documents how the system behaves when a resident completes a communal task. This particular use case was selected because it represents one of the most complex workflows in the system, involving multiple components, business rules and data updates (see Appendix I).

The documented sequence describes the process that occurs when a user completes a communal task through the user interface. The operation involves user interaction, controller logic, facade coordination, domain object updates, task logging, community point aggregation, and persistent

export. By documenting this sequence the diagram provides a clear overview of how responsibilities are distributed across the system.

The sequence begins when the user initiates the “Complete Task” action in the user interface. This interaction is handled by the CompleteTaskController, which acts as the entry point from the user interface layer. The controller does not perform any business logic itself, instead it delegates the operation to the Cloverville Facade by calling the completeTask method.

Upon receiving the request, the facade coordinates the required logic. It first retrieves the relevant Resident object from the ResidentList and the selected CommunalTask from the CommunalTaskList. Once these objects are obtained, the facade calculates the points awarded for the task, including any participation boost associated with the resident. The resident’s personal points are then updated accordingly.

The sequence diagram highlights the central role of the Cloverville Facade. All communication between controllers and the domain layer is mediated through the facade.
This demonstrates how a complex user interaction is handled in a structured an maintainable way. By clearly separating user interface handling, application logic and domain logic, the system achieves both clarity and extensibility. The diagram provides insight into the runtime behaviour of the application and complements the class diagram by illustrating how the defined structures interact during execution.

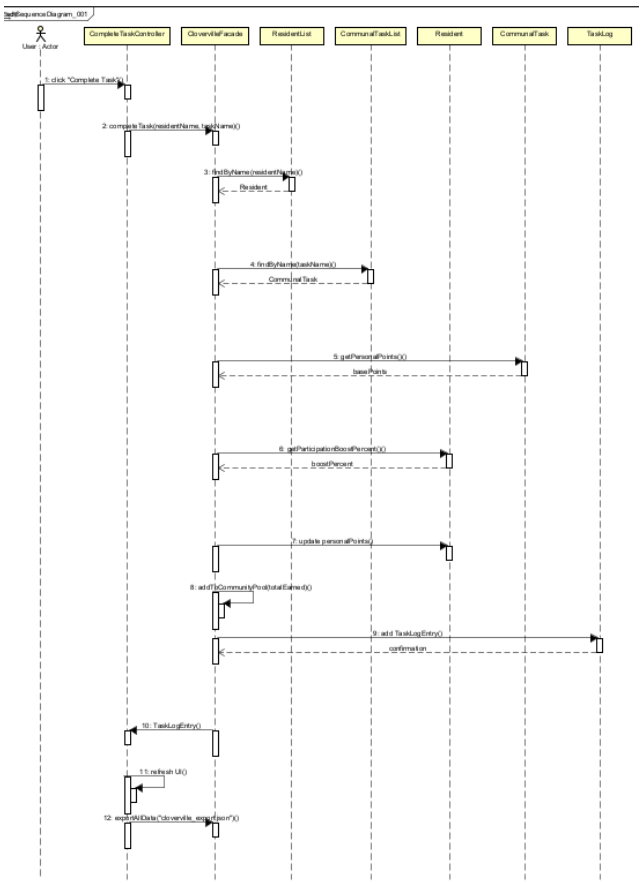

Figure 9. Sequence Diagram

3.2.3. Custom App Design

The Cloverville application was designed with a strong focus on clarity, modularity, and maintainability. The design process began by identifying the core concepts of the system, such as residents, communal tasks, trade offers, and green actions. These concepts were modelled as domain classes and visualised using a UML Class Diagram to ensure a clear and consistent structure before implementation.

A central design decision was the use of the Facade design pattern. The Cloverville Facade acts as the main coordination point for all application logic, allowing user interface controllers to remain lightweight and focused solely on handling user interactions. This approach reduced coupling between the user interface and the underlying logic, making the application easier to extend and maintain.

Each domain concept is managed through a corresponding list class, such as ResidentList, CommunalTaskList, and TradeOfferList. The design encapsulates collection management and supports a clear separation between data storage and application logic. Composition was used to represent ownership relationships, with the facade owning all list classes and controlling their lifecycle.

The class diagram was not only used as a planning tool but also as a reference throughout development. As features evolved, the diagram was updated to reflect changes in the system, ensuring consistency between design and implementation. This iterative approach helped maintain a coherent structure as the application grew in complexity.

3.2.4. Custom Website Design

The website prototype for Cloverville was designed using Figma and served as a visual foundation for the implementation phase. Since the website is publicly accessible and aimed at both residents and visitors, particular attention was given to creating an inspirational, eco-friendly, and user-friendly design that reflects Cloverville's sustainable values. A green and natural color palette was chosen to align with the ecological theme of the project and to create a calm and welcoming atmosphere.

The website consists of several pages that directly reflect the defined requirements. The main page acts as a landing page and includes a hero section introducing Cloverville, followed by integrated About Us and Contact sections to present the community's philosophy and provide relevant information. Additional pages include Trade Offers, Green Actions, and Weekly Focus Tasks, allowing users to explore current activities, inspiration, and opportunities within the community. The structure of the website was designed to be clear and intuitive, ensuring that information is easy to locate (see Appendix J).

Figure 10. Figma Hi-Fi Prototype of the Website

3.3. Implementation

3.3.1. View

The view layer of the Cloverville application is responsible for presenting the graphical user interface and handling all user interactions. The interface was implemented using JavaFX, which was chosen due to its strong support for modern UI components, scene management and event-driven programming.

Each screen of the application is defined using FXML files, created and visually adjusted with SceneBuilder. This approach allowed the separation of layout design from application logic, improving readability and maintainability. Controllers associated with each FXML file handle user input such as button clicks, sections and form submissions.

The user interface includes views for managing residents, communal tasks, trade offers, green actions, and task completion as well as a main menu displaying global information such as the community pool.

3.3.2. Scene Management

Navigation between views is handled by a dedicated SceneManager class. This class stores the primary application stage and is responsible for loading and switching scenes at runtime.

By centralizing scene switching in a single class, the application avoids duplicated code and ensures consistent navigation behavior across all controllers. Each controller can request a scene change by calling the appropriate method in the SceneManager.

3.3.3. Model and Application Logic

The application logic and data handling are implemented în the model layer. At the core of this layer lies the Cloverville Facade, which acts as a single entry point to the system's functionality.

The facade follows the Facade design pattern, providing simplified methods for operations such as:

- Creating and deleting residents
- Managing communal tasks and trade offers
- Updating green points and the community pool

- Exporting and importing application data

Controllers communicate only with the facade, ensuring that the view layer remains independent of the internal data structure and business logic.

3.3.4. Data Structures and Lists

To organize data efficiently, the application uses multiple list classes, including:

- ResidentList
- CommunalTaskList
- TradeOfferList
- GreenActionList

Each list class encapsulates a collection of domain objects and provides methods for adding, removing and retrieving elements. This structure improves modularity and aligns with object-oriented design principles.

3.3.5. Task Completion and Participation Boost

When a resident completes a communal task, the system calculates the total points earned by combining the base task points with the resident's participation boost percentage. This calculation is handled entirely within the facade to ensure consistency.

```java
public TaskLogEntry completeTask(String residentName, String taskName) {  1 usage  marcusneb

    Resident r = residentList.findByName(residentName);
    CommunalTask t = taskList.findByName(taskName);

    if (r == null || t == null) return null;

    int base = t.getPersonalPoints();
    int boostPercent = r.getParticipationBoostPercent();
    int bonus = (int) Math.round(base * (boostPercent / 100.0));

    int totalEarned = base + bonus;

    r.setPersonalPoints(r.getPersonalPoints() + totalEarned);
    addToCommunityPool(totalEarned);

    TaskLogEntry entry = new TaskLogEntry(r, t);
    TaskLog.log.add(entry);

    return entry;
}
```

Figure 11. Task Completion and Participation Boost Code

3.3.6. File Persistence (JSON Export & Import)

Cloverville uses file-based persistence implemented through JSON serialization. The system does not rely on a database; instead all application data is stored în a single JSON file.

The facade provides two key methods:

1. exportAllData() - serializes all data into a JSON file

```java
public void exportAllData(String filename) {  13 usages  marcusneb *


    Path exportPath = Paths.get(
            first: "C:/Users/marcu/Documents/GitHub/cloverville_website",
            filename
    );

    Gson gson = new GsonBuilder().setPrettyPrinting().create();
    Map<String, Object> root = new HashMap<>();

    // Community Pool
    root.put("communityPool", communityPool);

    // Residents
    List<Map<String, Object>> residentsJson = new ArrayList<>();
    for (Resident r : residentList.getAllResidents()) {
        Map<String, Object> map = new HashMap<>();
        map.put("name", r.getName());
        map.put("personalPoints", r.getPersonalPoints());
        map.put("greenPoints", r.getGreenPoints());
        map.put("participationBoostPercent", r.getParticipationBoostPercent());
        residentsJson.add(map);
    }
    root.put("residents", residentsJson);
```

Figure 12. exportAllData() Method

2. importAllData() - loads the JSON file at application startup and reconstructs the internal state

```java
public void importAllData(String filename) {  1 usage  marcusneb
    try {
        Path importPath = Paths.get(System.getProperty("user.dir"), filename);

        if (!Files.exists(importPath)) {
            System.out.println("No JSON file → starting fresh.");
            return;
        }

        String json = Files.readString(importPath);
        Gson gson = new Gson();
        Map<String, Object> root = gson.fromJson(json, Map.class);

        // Community Pool
        this.communityPool = ((Double) root.getOrDefault( key: "communityPool",  defaultValue: 0.0)).intValue();

        // Residents
        residentList.clear();
        List<Map<String, Object>> residentsJson = (List<Map<String, Object>>) root.get("residents");
        if (residentsJson != null) {
            for (Map<String, Object> map : residentsJson) {
                Resident r = new Resident(
                        (String) map.get("name"),
```

Figure 13. importAllData() Method

This approach ensures that data is preserved between application runs, exported file can be reused by the website without modification and the system remains lightweight and easy to maintain.

The export operation is automatically triggered after any meaningful data changes, ensuring that the JSON file always reflects the latest system state.

3.3.7. Website Integration

The exported JSON file is designed to be directly consumable by the Cloverville website.

Using JavaScript, the website reads the JSON file and dynamically displays trade offers, communal tasks, green actions, and community pool value.

This creates a seamless connection between JavaFX application and the static website, allowing data managed by administrators to be reflected immediately on the public-facing pages.

3.3.8 User Guide and Usability Support

To support usability and ensure that the system can be operated by users with limited technical experience, a user guide was created for the Cloverville administration application. The guide explains the main system functionalities step by step, including managing residents, registering tasks and green actions, handling points, and publishing website data(see Appendix K).

The user guide is designed to be clear and concise, using simple language and screenshots where appropriate. This aligns with the project requirement that the system should be accessible to Green Bob, who is not comfortable using complex digital systems. By providing written guidance

alongside an intuitive interface, the system reduces the learning curve and supports independent use without continuous technical assistance.

3.3.9. Summary

The Cloverville implementation combines JavaFX for the user interface, a Facade-based architecture for clean logic separation, and a JSON-based file persistence for data storage and website integration. The design results in a modular, maintainable and extensible system that fulfills all project requirements while remaining simple and effective.

3.4. Testing

Testing was conducted to verify that the system functions according to the defined use cases and requirements. Each test case was derived directly from the implemented use cases to ensure traceability between requirements, system behavior, and test results.

The JavaFX administration application was tested by executing each use case and observing whether the expected behavior occurred. This included creating, editing, and deleting residents, communal tasks, and trade offers, as well as registering communal participation, green actions, participation boosts, and personal point resets. Special attention was given to verifying correct point assignment, anonymity of green actions, and system consistency after updates.

In addition, testing confirmed that the JSON export functionality worked correctly. After changes were made in the administration application, the generated JSON files were inspected and validated. The website was then tested to ensure that it correctly fetched and displayed the updated data, including trade offers, community points, weekly focus tasks, and anonymous green actions.

Due to time constraints and the scope of the project, testing was limited to manual use-case and requirement-based testing and did not include automated tests or real end-user validation.

Use Case	**Result**	**Comments**
Manage Residents: Add residents	Passed	Resident can be added to the system.
Manage Residents: Edit Resident Information	Passed	Existing resident information can be edited in the system.
Manage Residents: Delete Resident	Passed	Residents can be removed from the system.
Manage Communal Tasks: Add Communal Task	Passed	Communal tasks can be added to the system.
Manage Communal Tasks: Edit Communal Task	Passed	Existing communal tasks can be edited in the system.
Manage Communal Tasks: Delete Communal Task	Passed	Existing communal tasks can be removed from the system.
Manage Trade Offers: Add Trade Offer	Passed	Trade offers can be added to the system.

Manage Trade Offers: Edit Trade Offer	Passed	Existing trade offers can be edited in the system.
Manage Trade Offers: Delete Trade Offer	Passed	Trade offers can be removed from the system.
Register Communal Participation	Passed	Participation can be registered and personal points are assigned.
Register Green Action	Passed	Green actions can be registered and shown anonymously.
Publish Website Data	Passed	Updated data can be exported to JSON for the website.
Create Participation Boost	Passed	Participation boost settings can be configured
Reset Personal Points	Passed	Personal points can be reset and unused points transferred.
View Website Content	Passed	Website pages display updated content from JSON files.

Figure 14. Tests

4. Results and Discussion

4.1. Results

In addition to use case testing, the system was evaluated through requirement-based testing to ensure that all defined requirements (01–19) were fulfilled. Each requirement was tested by verifying that the implemented functionality behaves according to the specification defined during the analysis phase.

The tests focused on confirming that core system features, such as resident management, communal tasks, trade offers, point calculations, and website data publishing, operate correctly and consistently. Functional requirements related to data creation, updates, deletion, and rule enforcement were validated directly through interaction with the JavaFX administration application.

Requirements	**Test**	**Comments**
01 – Create Resident Records	Passed	Resident details are correctly stored and shown in the system.
02 – Manage Resident Information	Passed	Resident information updates correctly.
03 – Remove Residents	Passed	Only valid residents remain available in the system.

04 – Manage Communal Tasks	Passed	Communal tasks are stored and displayed correctly.
05 – Create Trade Offers	Passed	Trade offers are successfully added to the system.
06 – Update Trade Offers	Passed	Trade offer details update correctly.
07 – Remove Trade Offers	Passed	Outdated offers are removed from the system.
08 – Register Communal Participation	Passed	Personal points are assigned to residents correctly.
09 – Register Green Actions Anonymously	Passed	Green actions are stored without exposing resident identity.
10 – Assigning Community Points	Passed	Community points increase based on defined rules.
11 – Publish Website Data	Passed	Website displays up-to-date information from JSON files.
12 – Define Community Point Rules	Passed	Updated rules are applied when actions are registered.
13 – View Trade Offers	Passed	Trade offers are visible to residents and visitors.
14 – View Weekly Green Focus Tasks	Passed	Weekly tasks are displayed correctly.
15 – View Community Green Points	Passed	Total points and milestone progress are shown.
16 – Browse Green Actions	Passed	Green actions are shown as an inspiration catalogue.
17 – Configure Participation Boost	Passed	Boost is applied according to defined conditions.
18 – Auto-Transfer Unused Personal Points	Passed	Unused personal points are transferred to the community pool.
19 – Periodic Personal-Point Reset	Passed	Personal points are reset successfully
NFR1 – Technology Implementation	Passed	The administration system is implemented in JavaFX, and data is exported via JSON and displayed correctly on the HTML/CSS/JavaScript website.
NFR2 – Anonymity	Passed	Green actions displayed on the website

		do not contain resident names, identifiers, or contact information.

Figure 15. Results

4.2. Discussion

The Cloverville system was designed to support the administrative needs of the ecological village by enabling the management of residents, communal tasks, trade offers, green actions and the associated point systems. The primary objective was to provide a simple but structured way to document and coordinate the otherwise “invisible work” carried out by the community. The implemented solution fulfills these objectives through a clear and efficient design that supports both internal administration and public information sharing via a website.

A file-based persistence mechanism was selected as a key design decision. This approach allows for straightforward storage and retrieval of system data without introducing the complexity of a database solution. Given the limited data volume and the requirement that the system be operated locally by a small number of users, the file-based solution proved sufficient and reliable. It also supports the requirement of publishing selected data to a website in a lightweight and accessible format.

During the development process, particular challenges arose in the creation and refinement of UML diagrams, especially class and sequence diagrams. Accurately modeling the relationships and interactions between entities such as residents, communal tasks, trade offers, and point handling required careful analysis. Through iterative refinement, these diagrams were adjusted to better reflect the implemented architecture, which contributed to improved structural clarity and a more coherent system design.

Testing of the implemented system indicates that the core functionality operates as intended. Resident management, task registration, point assignment, trade offer handling, and data export behave consistently and according to the defined system rules. The system also successfully maintains data consistency across application restarts through its import and export mechanism. While the current implementation meets the project requirements, further improvements may be required if the system scope expands, such as increased automation, enhanced validation, or alternative persistence solutions.

5. Conclusion and Recommendations

The results demonstrate that the developed system fulfills all defined functional and non-functional requirements. Both use case testing and requirement-based testing confirmed that the JavaFX administration application operates correctly and consistently, enabling full management of community data. Additionally, the JSON-based integration between the administration application and the website functions as intended, allowing up-to-date information to be displayed dynamically without manual intervention. This confirms the effectiveness of the chosen architecture and data flow.

From a broader perspective, the project shows that lightweight, file-based solutions can be sufficient and reliable for small-scale, locally operated systems. By avoiding unnecessary technical complexity, the system remains easy to maintain and aligns with Cloverville's values of simplicity and sustainability. The separation between administrative functionality and public information sharing also supports transparency while respecting privacy, particularly through the anonymous handling of green actions.

Based on these findings, several recommendations can be made for future development. If the system were to be expanded, automated testing and stronger input validation could be introduced to further increase robustness. A database-backed persistence layer could be considered if data volume or concurrent access requirements grow. Future research or development projects could also explore usability testing with real users to evaluate how the system impacts long-term motivation and participation within the community.

Overall, the Cloverville project demonstrates how a carefully designed, requirement-driven software solution can effectively support community-based sustainability initiatives. The project provides a solid foundation that can be extended or adapted to similar small-scale ecological or cooperative environments.

6. References

Deci, E. L., & Ryan, R. M. (2000). Intrinsic and extrinsic motivations: Classic definitions and new directions. Contemporary Educational Psychology, 25(1), 54–67.

Limbu, S. (2024, May). Ecovillages from around the world for sustainable living. Earth.Org. https://earth.org/ecovillages-from-around-the-world-for-sustainable-living/

7. Appendices

Project Description:

\Cloverville-Group4\Appendices\Project Report\A. Project Description

Requirements:

\Cloverville-Group4\Appendices\Project Report\B. Requirements

Use Case Diagram:

\Cloverville-Group4\Appendices\Project Report\C. Use Case Diagram

Use Case Descriptions:

\Cloverville-Group4\Appendices\Project Report\D. Use Case Discriptions

Relation Between Requirements and Use Cases:

\Cloverville-Group4\Appendices\Project Report\E. Relation Between Requirements and Use Cases

Activity Diagram:

\Cloverville-Group4\Appendices\Project Report\F. Activity Diagram:

Domain Model:

\Cloverville-Group4\Appendices\Project Report\G. Domain Model

Class Diagram:

\Cloverville-Group4\Appendices\Project Report\H. Class Diagram

Sequence Diagram:

\Cloverville-Group4\Appendices\Project Report\I. Sequence Diagram

Website Wireframes:

\Cloverville-Group4\Appendices\Project Report\J. Website Wireframe

User Guide:

\Cloverville-Group4\Appendices\Project Report\K. User Guide

VIA University College

Software Technology Engineering.

1st Semester

Cloverville

Process Report

3 Students:

Diana Boiko (360245)

Marcus-David Muresan (361734)

Maciej Religa (363666)

Supervisors:

Line Lindhardt Egsgaard

Michael Viuff

Hand-in Date:

19.12.2025

Characters:

36243

Table of Content

1. Introduction **3**
2. Group Work **4**
2.1. Personal Profiles 4
2.2. Team Dynamics and Reflection 4
Team Development 6
Conflict Handling 7
Reflection on Teamwork 9
2.3. Team Type Diagram 9
3. Project Initiation **11**
Analysis and Early Challenges 12
Feedback 14
Guidance 14
Transition to Implementation 15
4. Project Execution **15**
5. Personal Reflections **18**
Learning Outcomes 18
6. Reflection on Supervision **21**
7. Conclusion **22**
8. References **24**

1. Introduction

This process report documents our work on the Cloverville project, developed as part of the Software Engineering Project (SEP1) at VIA University College. The project was carried out within the Problem-Based Learning (PBL) framework (Cornell University, 2025), which emphasizes student responsibility, collaboration, and learning through real-world problem situations. Through this approach, we were encouraged not only to develop a technical solution, but also to reflect on our learning process, teamwork, motivation, and methods used throughout the semester.

Our motivation for the project was shaped both by the technical theme and by the learning environment created through PBL. From a theoretical perspective, our experience can be understood using Maslow's Hierarchy of Needs (McLeod, 2025). Feeling accepted within the group and having a safe environment to ask questions and make mistakes contributed positively to our engagement and willingness to learn. As the project progressed, motivation increasingly shifted toward higher-level needs such as competence, recognition, and self-development.

Self-Determination Theory (Niemiec & Ryan, 2009) further explains our learning process by focusing on three core needs: autonomy, competence, and relatedness. The SEP1 structure supported autonomy by giving us freedom to plan our workflow, choose tools, and make design decisions. Competence was developed through acquiring new technical skills, such as JavaFX, UML modeling, and JSON-based data integration. Relatedness was supported through close collaboration, frequent communication, and offline group work, which strengthened our sense of responsibility toward each other and the project.

From the beginning of the semester, the project progressed through clear phases including analysis, design, implementation, and testing. Early stages focused on understanding the Cloverville case and translating an informal problem description into structured requirements and models. Later, during the intensive three-week development period, our focus shifted toward implementation and integration of the JavaFX application and website. Progress and decisions were documented through meetings, shared tools, and continuous reflection within the group.

This introduction provides a foundation for the following sections of the process report, which reflect on group work, project initiation, cooperation, challenges, and learning outcomes. By combining practical experience with relevant motivational and teamwork theories, this report aims to demonstrate not only what we built, but how and why we learned throughout the project.

2. Group Work

2.1. Personal Profiles

Diana Boiko (Ukraine)
Diana brought previous experience with JavaScript, Python, HTML, CSS, and digital design from her AP degree in Multimedia Design, where she also learned UX/UI principles and tools such as Figma. In the Cloverville project, she contributed mainly to the JavaFX application, website development, use case descriptions, requirements, and overall project organization, including planning meetings and coordinating tasks.

Her strengths included structured task organization, creative design skills, fast learning, and reliable coding. Throughout the project, she learned how to implement Java classes more effectively, work with system structure, and apply her prior JSON experience in a new context. She also developed stronger communication skills and became more confident in resolving team issues calmly and constructively.

Marcus-David Muresan (Romania)
Marcus contributed prior experience in programming, including HTML, CSS, JavaScript, and C++. He played a key role in the design and implementation of the system by developing class and sequence diagrams, shaping the overall application architecture, and implementing core Java logic. In addition, Marcus contributed to planning development steps, coordinating architectural decisions, and ensuring consistency between the system design and the codebase.
He strengthened his communication skills and gained experience working collaboratively in a group setting, including task coordination, deadline management, and adhering to group agreements and project planning.

Maciej Religa (Poland)
Maciej functioned as a reliable team member who focused on executing specific tasks assigned during group meetings. While he relied on the team for high-level architectural decisions, he was dedicated to delivering his parts of the project. His main technical contributions were the implementation of the "Green Actions" module in the Java application and finalizing the website. Specifically, Maciej ensured that the website content and structure strictly aligned with the project requirements and worked on the CSS to ensure the layout was responsive. He was a consistent contributor who prioritized completing his coding tasks and Astah diagrams to support the group's progress.

2.2. Team Dynamics and Reflection

Our project group consisted of three members from different cultural and educational backgrounds: Diana (Ukraine), Marcus (Romania), and Maciej (Poland). The group was assigned randomly at the beginning of the semester. Although most SEP1 groups consisted of four members, all three active members in our group participated consistently throughout the semester.

To reflect on how our group organized collaboration and task distribution, we used team structure models introduced during the course, including parallel, convergence, chain, and mesh structures. In the early stages of the project, our work resembled a parallel structure, where tasks were divided clearly and handled individually. As the project progressed and technical dependencies increased, our collaboration shifted toward a convergence and mesh structure, especially during implementation and debugging phases.

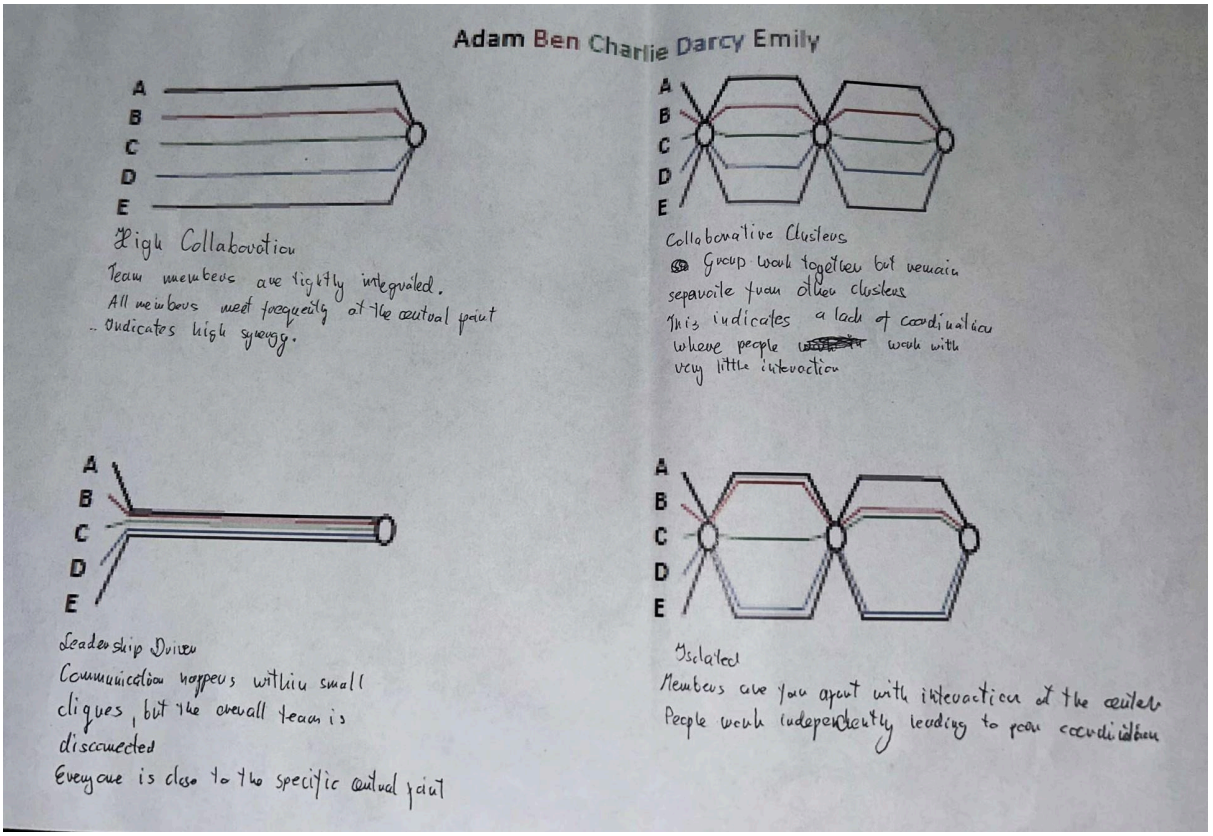

Figure 1. Team structure models

This transition allowed for better knowledge sharing, faster problem solving, and increased mutual support, which was particularly important given that our group consisted of three members. The drawings illustrate how our teamwork evolved from individual responsibility toward stronger collaboration as the project complexity increased.

Motivation and Learning

From a motivational perspective, our group environment supported several key aspects of both Maslow's Hierarchy of Needs (McLeod, 2025) and Self-Determination Theory (SDT) (Niemiec & Ryan, 2009). We felt safe asking questions, making mistakes, and admitting uncertainty, fulfilling Maslow's need for safety. Working in a small group also strengthened our sense of belonging, as communication was direct and personal, which supported Maslow's social needs.

According to SDT, motivation is influenced by autonomy, competence, and relatedness. All three needs were present in our group. We had autonomy in organizing our workflow, setting internal deadlines, and deciding how to approach tasks. Competence developed steadily as we improved our ability to transform UML diagrams into structured Java code and connect

application logic to JSON-based data exchange. Relatedness increased as the project progressed; communication became more open, and discussions felt more comfortable and productive over time.

In addition to Self-Determination Theory, the ABC model helped explain how our motivation and well-being were maintained during the project. Regular active participation (*Act*), supportive collaboration (*Belong*), and commitment to a meaningful project (*Commit*) contributed to sustained engagement, even during demanding phases of the project (University of Copenhagen, n.d.).

Motivation within the group was a mix of learning-driven and deadline-driven factors. While we were genuinely interested in learning new technologies and improving our technical skills, we were also aware of time constraints and report-writing requirements. This balance created a realistic working environment that reflected both intrinsic and extrinsic motivation.

Team Development

Our group development aligns well with Tuckman's stages of team development (MIT Human Resources, n.d.). During the forming stage, the group was motivated and positive, focusing on understanding the project and each other's working styles. In the storming phase, challenges emerged mainly around system structure, especially when designing the class diagram and deciding how to implement it sustainably. These challenges did not escalate into conflict but required discussion and clarification.

As we entered the norming stage, collaboration became smoother. Responsibilities were clearer, communication improved, and trust developed as we saw consistent contributions from all members. In the performing stage, particularly during the three-week implementation period, the group worked efficiently and collaboratively, solving problems quickly and supporting each other across technical and documentation tasks.

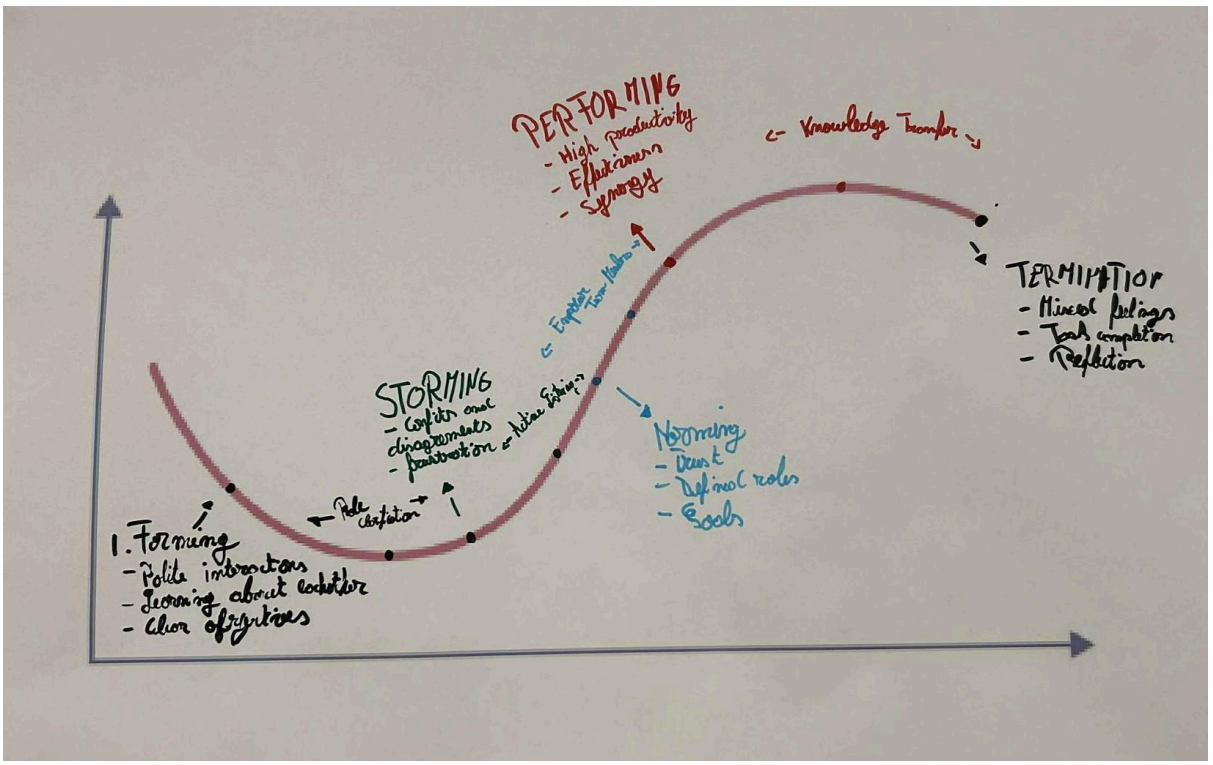

Figure 2. Visualization of Tuckman's model

Conflict Handling

Conflicts in our group remained minimal and stayed at early stages of Glasl's Conflict Ladder (Jordan, n.d.). Minor misunderstandings typically involved task interpretation or timing and were resolved through calm discussion and clarification. We mainly used a collaborative approach, consistent with the Dual Concern Theory (Kelley School of Business Executive Education, 2024), balancing concern for individual workload with concern for group success.

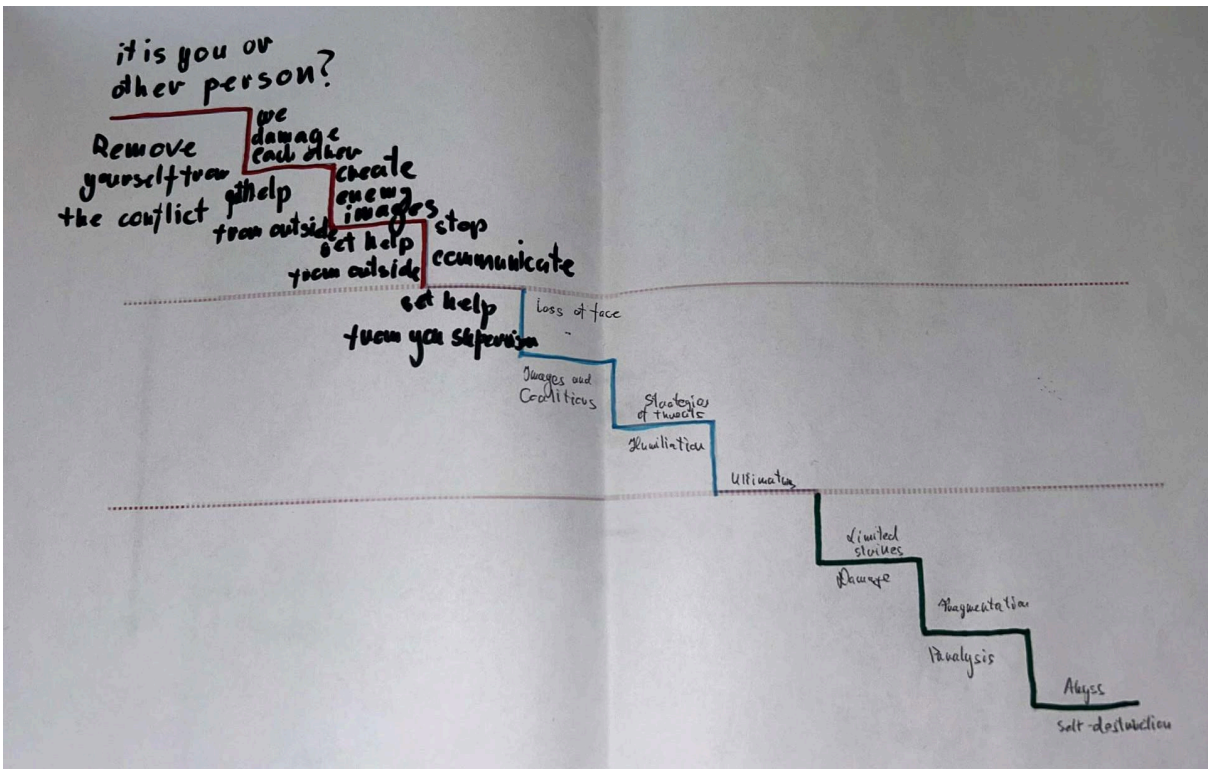

Figure 3. Glasl's Conflict Escalation Ladder

Social loafing was largely avoided (Hoffman, 2023). All members contributed actively, and responsibilities were clearly defined using Trello (Jenkins, 2021). When minor delays occurred, they were communicated openly, and the group responded with reminders and adjustments rather than criticism. The small group size made individual contributions more visible, which helped maintain accountability and engagement.

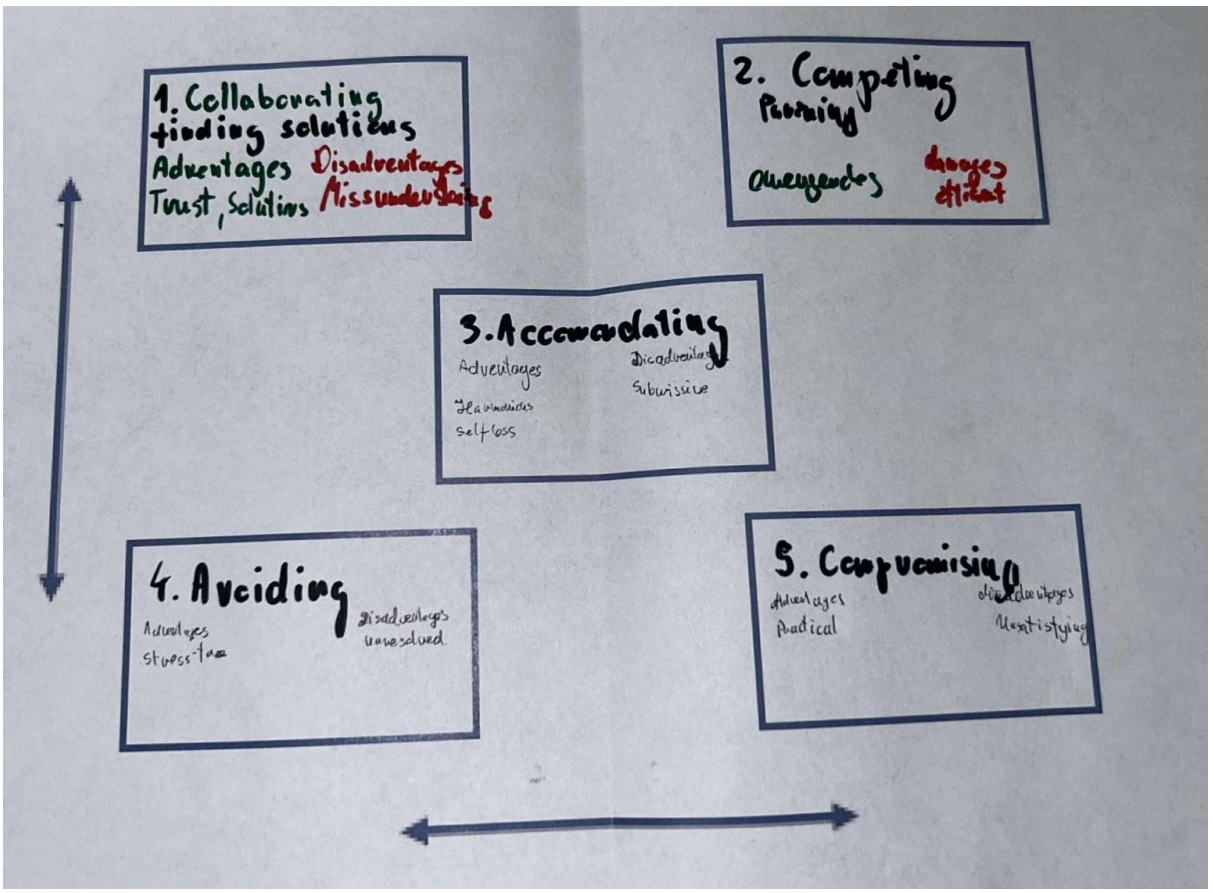

Figure 4. Dual Concern Model

Reflection on Teamwork

Overall, working in a smaller group proved to be an advantage rather than a limitation. Communication was efficient, coordination was clear, and decision-making was faster. Although an additional member could have reduced workload during report writing, the strong collaboration, mutual support, and shared responsibility allowed us to complete the project successfully. This experience reinforced the importance of trust, communication, and structured collaboration in problem-based learning environments(Cornell University, 2025).

2.3. Team Type Diagram

The team type diagram illustrates how each group member perceives the balance between task focus and social interaction within the team. Although the positions of the individual members differ slightly, all three are placed within Type A: an efficient and well-functioning team. Our group placed strong emphasis on completing tasks efficiently, meeting deadlines, and maintaining a clear structure throughout the project. At the same time, open communication, mutual support, and a friendly atmosphere contributed to a sense of trust and cooperation within the team. The small differences in placement reflect individual working styles rather than fundamental disagreements about how the team functioned.

Overall, we are satisfied with our position in the Type A quadrant. Being a small group required a high level of responsibility from each member, and this diagram confirms that we managed to combine efficiency with good collaboration.(see Appendix A)

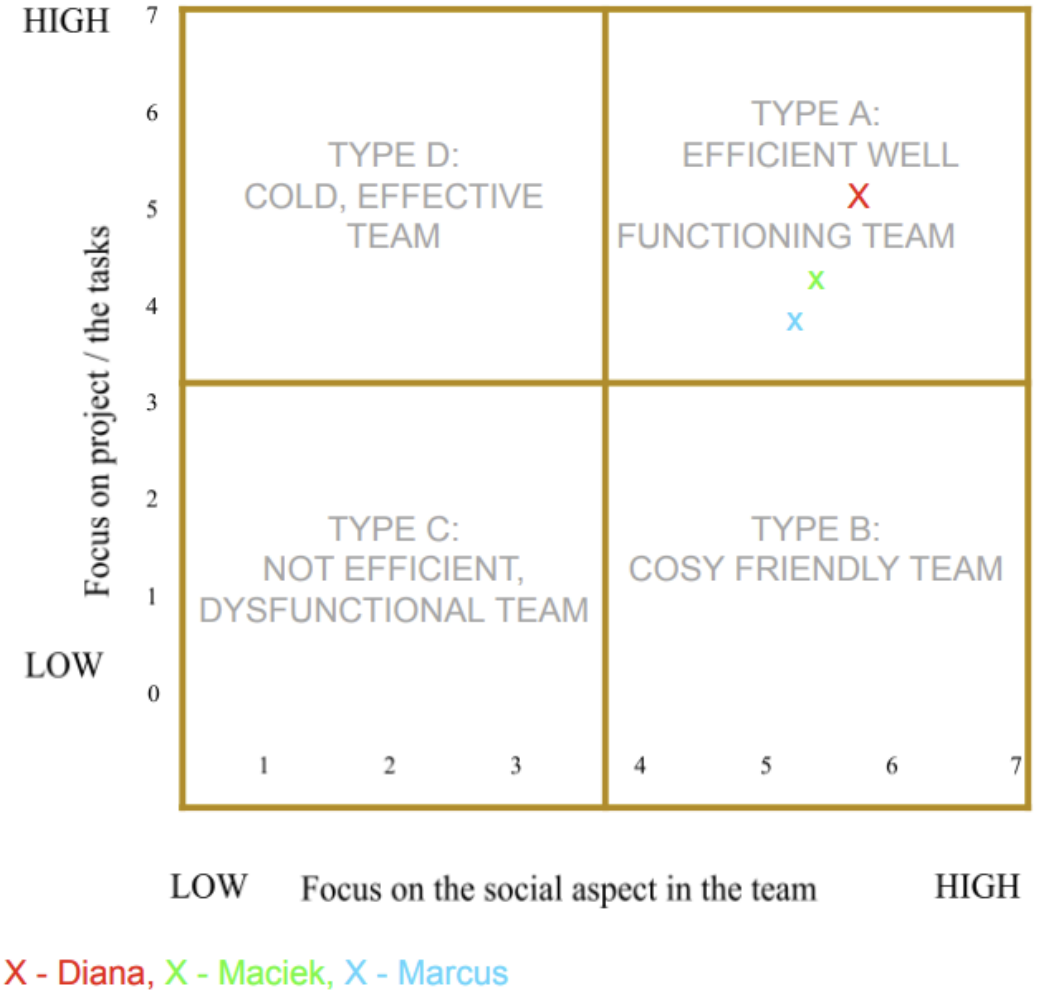

Figure 5. Team Type Diagram

3. Project Initiation

The Cloverville project was introduced to us through an interview-style case description with Green Bob, the village coordinator. This format initially made the problem domain feel approachable and realistic, as it clearly described the daily challenges of managing an ecological community using a handwritten notebook. However, as we began to analyze the case more deeply, we realized that transforming this informal narrative into a structured software solution required careful interpretation and many design decisions.

Group Contract

At the beginning of the project initiation phase, our group established a group contract to define expectations regarding cooperation, communication, meetings, and responsibility (see Appendix B). The contract outlined how often we would meet, how progress should be reported, how decisions should be made, and how potential conflicts or absence should be handled. By agreeing on clear rules early on, the group created a shared framework that supported trust, accountability, and structured collaboration throughout the project. The group contract also served as a reference point during the project when expectations needed clarification, especially given the increased workload of working as a group of three, and it did not require revision as emerging issues were resolved through open communication.

Rich Picture

To better understand the problem domain and identify key elements such as actors, data flows, and responsibilities, we created a rich picture early in the initiation phase. This visual representation helped us clarify how residents, green actions, communal tasks, trade offers, and community points relate to Green Bob and to each other. The rich picture played an important role in aligning our understanding as a group and served as a bridge between the narrative case and the more formal requirement analysis that followed(see Appendix C).

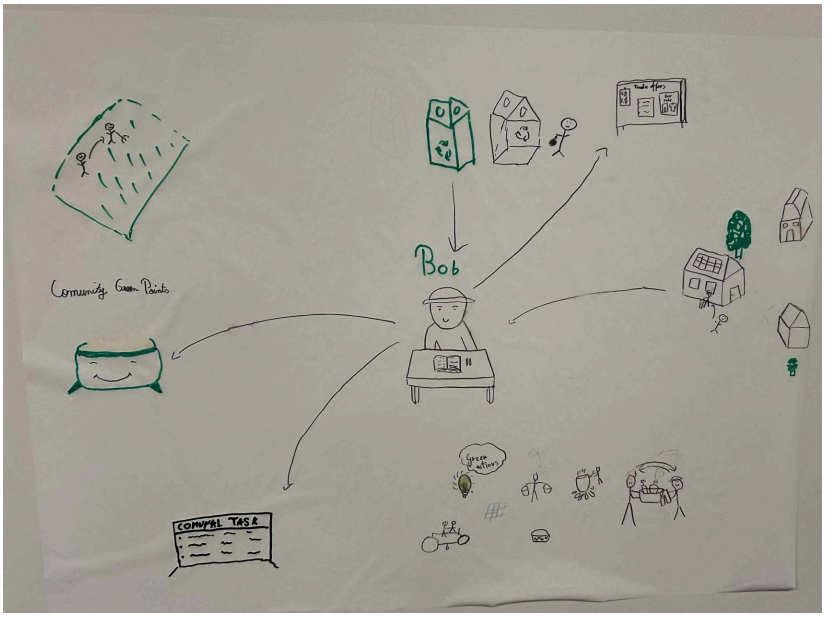

Figure 6. Rich Picture

Project Description

The project description played a central role in structuring our work and aligning the group around a shared understanding of goals, scope, and working methods. After analysing the Cloverville case and creating a rich picture, we used the project description to translate the informal interview with Green Bob into a clear and manageable plan. This included defining the purpose of the system, identifying key stakeholders, outlining our main milestones, and setting delimitations to avoid unnecessary complexity.

Working on the project description helped us clarify what the system should and should not include, which was especially important given the limited time frame and our group size of three. It also forced us to reflect on feasibility, ethical considerations such as anonymity and fairness, and the balance between technical ambition and realistic implementation. By documenting these decisions early, the project description provided a stable reference point throughout the semester and supported more focused decision-making during later phases of analysis, design, and implementation. Overall, this phase helped us establish a strong foundation for the project and reduced uncertainty in the subsequent stages.

Analysis and Early Challenges

During the initiation phase, our work followed a Waterfall development approach, as recommended in the SEP1 guidelines. We moved sequentially through analysis, design, and implementation, and only began coding after completing and revising requirements, diagrams, and system structure. While this approach sometimes made the analysis phase feel long, it helped us reduce uncertainty before implementation and prevented major structural changes later in the project. Especially as a group of three, having a clear phase-based workflow supported coordination, responsibility, and alignment with supervisor expectations.

One of the challenges was understanding and defining the points logic of the system. The case description did not provide a strict model for how personal points, community points, boosts, and resets should interact. To address this, we created our own point-flow scheme, even though this was not explicitly required in the curriculum. This extra step helped us reason about fairness, motivation, and system consistency, and later proved valuable during implementation.

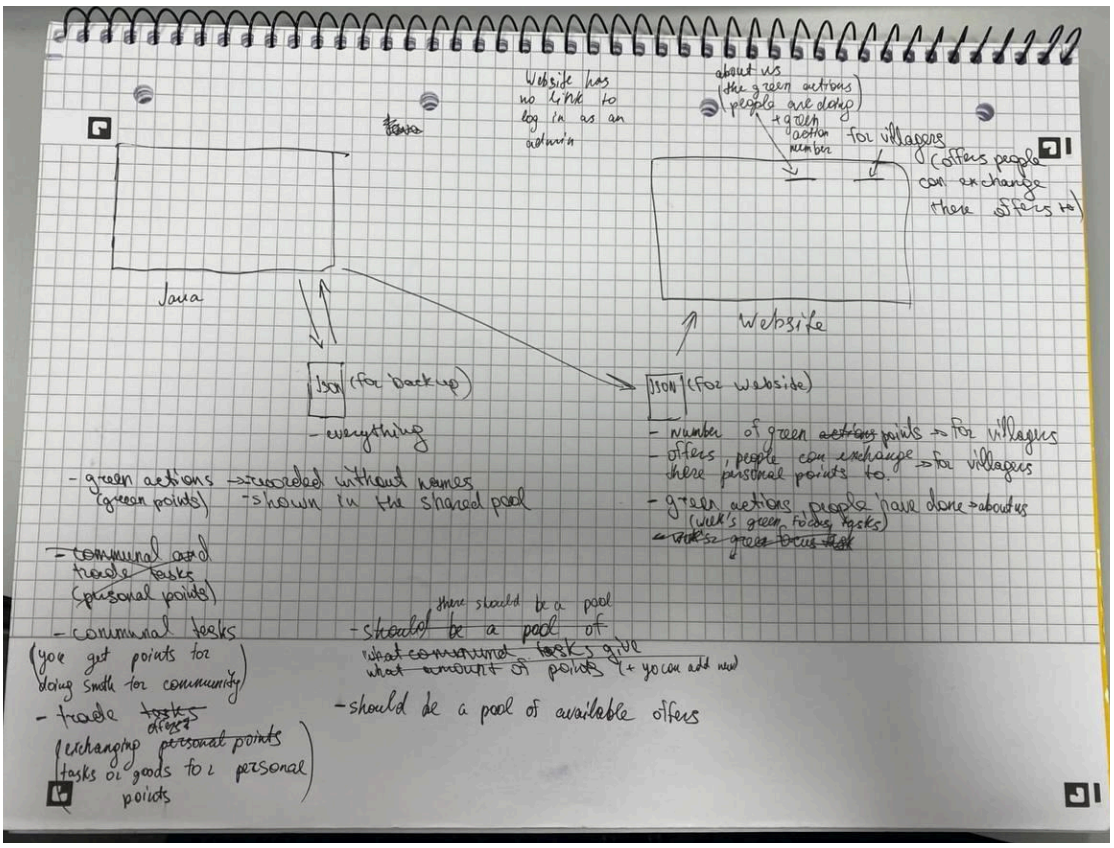

Figure 7. Project scheme

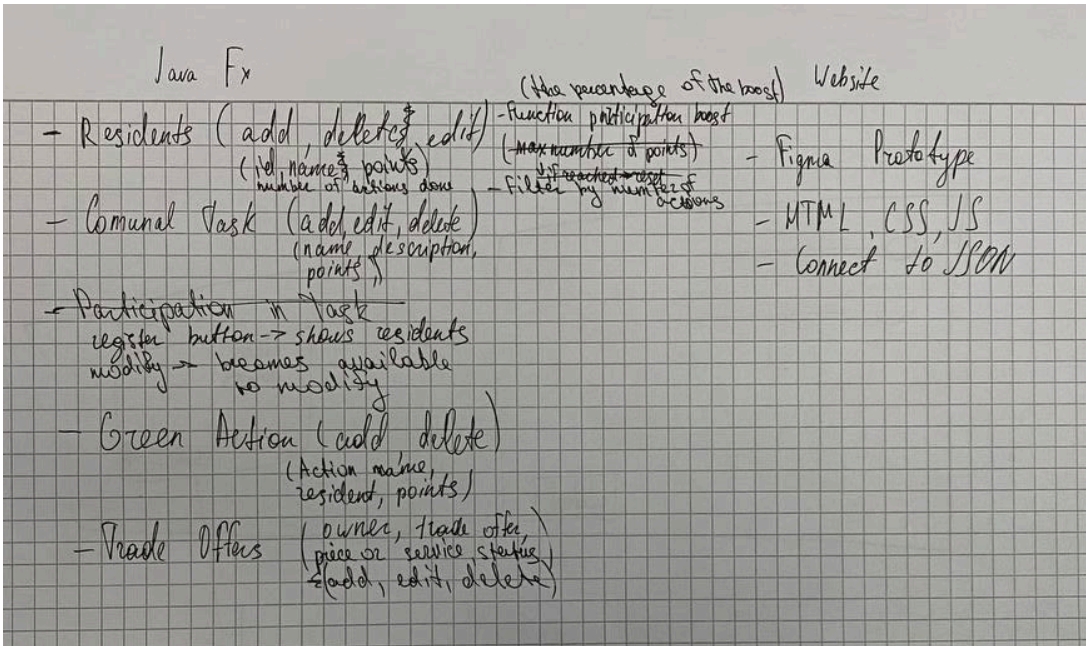

Figure 8. JSON data scheme

A further source of uncertainty was the separation between the admin application and the website. Through sketches and system diagrams, we clarified that the JavaFX application should function as a closed administrative tool for Green Bob, while the website should only

display selected, read-only data via JSON files. This separation supported both usability and ethical considerations such as data privacy and access control.

Feedback

Several factors helped us reduce uncertainty during the initiation phase. UML diagrams played a central role: use case diagrams, activity diagrams, and the domain mode allowed us to translate abstract ideas into concrete structures. Supervisor feedback was especially important during this phase, as it guided us toward clearer requirements, better system boundaries, and more realistic assumptions.

We also communicated actively with other project groups to understand how they interpreted feedback and requirements. This helped us validate our own decisions and gain confidence that we were moving in the right direction.

Guidance

From a leadership perspective, the initiation phase reflected elements of Theory Z / laissez-faire leadership (Cherry, 2025). While the overall framework and deliverables were clearly defined by the course, we were given trust in how we structured our work and interpreted the case. This balance between guidance and freedom encouraged responsibility and ownership, especially important in a small group of three where each member carried a significant workload.

Although we did not have full freedom in choosing tools or methodologies, the clear guidelines reduced ambiguity and helped us stay focused. Within those boundaries, we had autonomy in task distribution, internal deadlines, and collaboration style, which supported motivation and efficiency.

Transition to Implementation

Once the analysis and design phases were completed and approved, we entered the dedicated three-week implementation period. At this point, our early investment in understanding the problem domain paid off. Using Trello as our main planning tool, we organized tasks by individual responsibility and tracked progress daily through short check-ins(Jenkins, 2021). Being a group of three meant taking on larger responsibilities, but with clear communication and mutual support, we were able to stay mostly on track.

4. Project Execution

The execution of the Cloverville project followed a structured and phased approach, reflecting both the academic guidelines and our practical working style. The project progressed through planning, analysis and design, implementation, and testing. Throughout these phases, we continuously reflected on our methods, adjusted our approach, and incorporated feedback from supervisors to ensure steady progress and alignment with the project objectives.

Planning and Organization

During the implementation period, our group worked almost entirely offline and met every weekday in person. Each working day started with a short morning meeting, where we discussed what had been completed the day before, what tasks were still pending, and how the workload should be distributed for the day. We used Trello as our primary planning and tracking tool, with individual columns for each group member and a shared “What’s left” column. Tasks were moved and marked continuously, which helped maintain transparency and accountability(Jenkins, 2021).

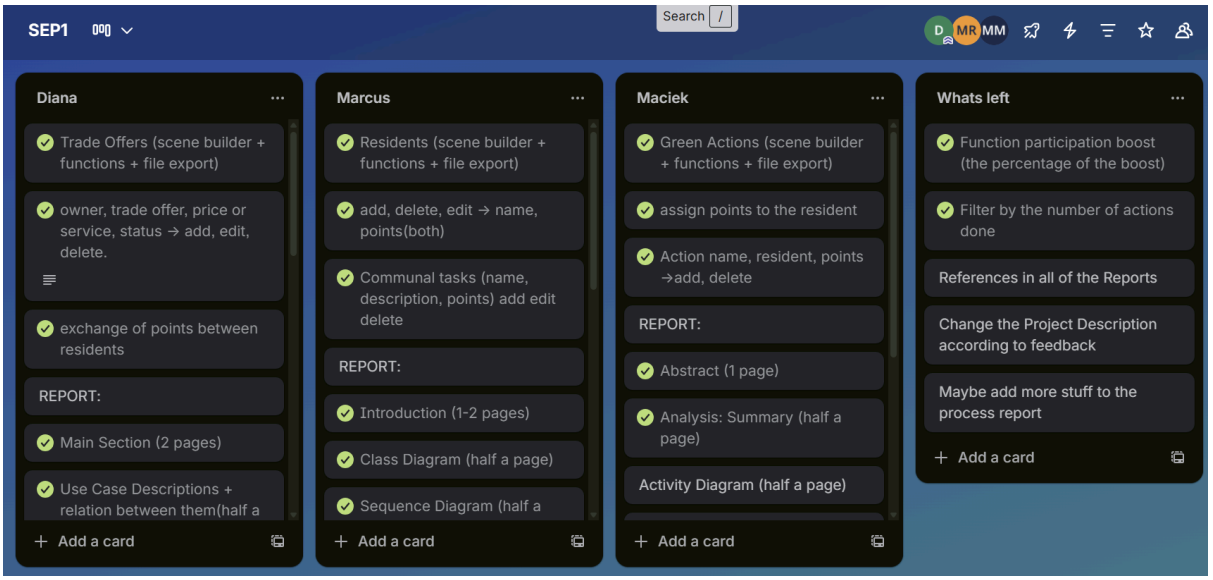

Figure 9. Trello Planning Board

Being a group of three meant that each member had a relatively large responsibility. While this occasionally created pressure, the clear structure and daily coordination helped us stay on track. Our way of working reflects elements of Theory Z (laissez-faire leadership) (Cherry, 2025), as we were trusted to organize our own time and tasks within clearly defined project boundaries. This autonomy increased our sense of ownership and responsibility toward the project.

Analysis and Design Phase

The analysis and design phase was one of the most demanding parts of the project. We produced all required analytical artifacts, including functional and non-functional requirements, use case diagrams and descriptions, activity diagrams, sequence diagrams, a domain model, and a class diagram. The requirements were rewritten three times, as initial versions were either too vague or did not fully align with the Cloverville case. Supervisor feedback played a crucial role in refining them into a form that was both realistic and implementable.

The class diagram proved to be the most difficult element, as it required multiple iterations before we found a structure that could realistically be translated into code. At one point, changes in the class diagram also required us to restructure parts of the implementation later on.

This phase followed a Waterfall approach, where we deliberately postponed coding until the analysis and design were sufficiently clear. Although this initially felt slow, it later proved beneficial, as it reduced confusion during implementation and strengthened our understanding of the system as a whole.

Implementation Phase

The implementation phase focused on turning the design into a working system. The solution consists of two main components:

- A JavaFX administration application, implementing CRUD operations for residents, communal tasks, trade offers, green actions, and point management, as well as exporting data to JSON files.
- A website built with HTML, CSS, and JavaScript, which dynamically loads and displays data from the JSON files using fetch functionality.

The technical workload was divided so that each group member was responsible for one main JavaFX page, while shared tasks such as JSON handling and system integration were solved collaboratively. Integration between the two components followed a clear flow: JavaFX application → JSON files → Website.

To support usability and ensure that the system can be operated independently by Green Bob, a user guide was created alongside the implementation. The guide explains the core functionalities of the JavaFX administration application step by step, including resident management, task registration, point handling, and data publication for the website.

The most significant learning outcome during this phase was learning how to translate the class diagram into working code, structure controllers and models properly, and maintain clean and readable code. Working with JSON as a data bridge between backend and frontend significantly improved our understanding of data flow and separation of concerns.

Testing and Validation

Testing was conducted manually, as required by the project scope. We tested the system based on use cases and requirements, verifying that each function behaved as expected. This included checking CRUD operations, point calculations, anonymity of green actions, JSON file updates, and correct display of data on the website.

Testing showed that the system functioned correctly and that changes made in the JavaFX application were immediately reflected on the website through updated JSON files. While the lack of automated testing and real users is a limitation, the testing approach was sufficient for validating a functional prototype within the given timeframe.

Results and Limitations

We are particularly proud of the JSON integration, the overall system structure, and the improvement of our Java skills, especially regarding clean code and architectural thinking. The final solution fulfills the main objectives of the Cloverville case and demonstrates a clear connection between analysis, design, and implementation.

However, the project also had limitations. Time constraints, being a group of three, and writing a full project and process report for the first time increased the workload significantly. Despite this, the structured execution, strong communication, and continuous reflection allowed us to complete the project successfully.

5. Personal Reflections

Diana Boiko (Ukraine)

Learning Outcomes

During the Cloverville project, I experienced significant technical learning, particularly in JavaFX development, system structuring, and JSON handling. One of my main learning outcomes was improving my ability to translate abstract models, such as class diagrams, into working code. This process required several iterations and supervisor feedback, but it helped me develop a clearer understanding of controllers, models, and data flow. Working with JSON as the connection between the Java application and the website further strengthened my system-thinking skills and attention to structure and consistency. These experiences increased my sense of competence, as described in Self-Determination Theory (Niemiec & Ryan, 2009).

Collaboration Experiences

From a collaboration perspective, working in a small group of three influenced my learning positively. I often contributed by organizing tasks in Trello, coordinating progress, and supporting communication within the group. Although I was not a formal leader, this responsibility helped me improve my planning and communication skills. According to Maslow's hierarchy of needs (McLeod, 2025), a sense of belonging and psychological safety is essential for motivation. I felt comfortable asking questions, admitting uncertainty, and making mistakes, which supported both my learning and engagement.
There were moments of pressure when deadlines were tight or tasks were delayed, as even small issues had a noticeable impact in a small team. These situations taught me the importance of calm communication and addressing challenges early.

PBL Context

Working within the Problem-Based Learning (PBL) framework (Cornell University, 2025) required me to take responsibility for my own learning and actively seek solutions rather than rely on predefined instructions. The open-ended nature of the Cloverville case encouraged exploration, discussion, and reflection throughout the project. My motivation was a combination of learning-driven and deadline-driven motivation, as I was genuinely interested in improving my technical skills while also being aware of assessment requirements and time constraints. This balance reflects how autonomy and external structure can coexist productively within PBL.

Future Plans

Reflecting forward, I aim to further develop my skills in software architecture and clean code, particularly by working more with design patterns and improving separation of concerns. I also want to become more confident during early system design phases to reduce the need for later restructuring. In future projects, I plan to continue applying structured planning, clear communication, and reflective learning strategies to support both my individual development and effective teamwork.

Marcus-David Muresan (Romania)
Learning outcomes

During the development of Cloverville, I gained a deeper understanding of the technical details that exist beneath the code itself. Planning our steps in advance and clearly defining what needed to be implemented made me more aware of my responsibilities and more careful about what I wrote and committed to the project. This was challenging for me at first, but it proved necessary for effective collaboration within the group. At the beginning of the project, I had no clear idea of how to develop both a website and a Java application and connect them in a meaningful way, as this was something I had never done before. However, through consistent effort, guidance from my teammates, and support from our supervisors, we were able to overcome these challenges and make the system work as intended. Overall, this experience significantly strengthened my technical skills while also improving my ability to work effectively as part of a team.

Collaboration experiences

At the beginning of SEP, most groups consisted of four or five members, and being part of a group of only three initially felt intimidating. I had concerns about how we would manage the workload and coordinate our tasks effectively. Despite these concerns, our group worked together very well. Although the workload was demanding, having a smaller group improved communication, as decisions could be made quickly without involving many people. I also felt comfortable asking for help when I struggled with certain tasks, knowing that my teammates were supportive and willing to assist. This created a positive working environment where questions and collaboration were encouraged. Overall, working in a smaller group strengthened both my confidence and my teamwork skills, and it contributed significantly to the success of the project.

PBL context

Problem-Based Learning (PBL) initially felt like a cliché concept to me (Cornell University, 2025). However, through actively working within the PBL framework, I learned that every decision and action requires clear justification. I could no longer implement solutions simply because they felt appropriate; instead, I had to take responsibility for my choices and be able to explain and defend them. This approach increased my awareness of each contribution I made to the project and encouraged me to think more critically about my work. Overall, PBL helped me develop a more reflective and responsible approach to both problem-solving and software development.

Future plans
For future projects, I want to be more organized and concise in how I work. I really enjoyed the planning and design parts of this project, so I'd like to keep developing those skills. I also want to avoid leaving documentation until the end and instead work on it continuously throughout the project.

Maciej Religa (Poland)

Learning outcomes This project was a great chance for me to learn from my teammates who had more experience. My main focus was making sure the website actually matched what was asked in the requirements, which gave me good practice with HTML and CSS details. I also improved my Java skills by building the "Green Actions" part of the app. Drawing the diagrams in Astah was also helpful it finally made me understand how the UML logic we talked about in meetings translates into real documentation.

Collaboration experiences I liked working in a group of three because communication was quick and simple. To be honest, I was happy that the others took charge of the complicated system planning. This let me focus on finishing specific tasks that were assigned to me. I realized that my best contribution was simply doing my part on time and trusting the others with the architecture. This kept things moving and saved us from unnecessary arguments.

PBL context Working with Problem-Based Learning showed me that you can't just start coding random things (Cornell University, 2025); you have to look at the requirements first. At the beginning, I just wanted to write code, but I learned that checking the list especially for the website was the only way to make sure Green Bob got what he needed. It stopped me from just "coding" and made me think about solving the actual problem.

Future plans Next time, I want to be confident enough to help plan the system architecture instead of just waiting for tasks. I plan to practice more difficult Java logic on my own so I understand better how the backend works. I also want to manage my time better so I can start writing parts of the report earlier, rather than leaving a lot of it for the end of the project.

6. Reflection on Supervision

Throughout the Cloverville project, supervision played an important role in shaping both our process and the final outcome. From a student perspective, the supervision was primarily constructive and guidance-oriented rather than directive. Our supervisors provided feedback at key stages of the project, especially during the analysis and design phases, which helped us clarify misunderstandings and improve the quality of our work.

One of the most valuable aspects of the supervision was the clear and concrete feedback we received on requirements, use cases, and diagrams. Our requirements, for example, were revised multiple times based on supervisor input. Although this initially felt time-consuming, it significantly improved our understanding of the problem domain and helped ensure that the system we designed was realistic and implementable. In this sense, supervision directly influenced the structure and consistency of the project.

The supervision style reflected a trust-based approach, similar to McGregor's Theory Y (Hancock, 2025), where we were given autonomy to make decisions within a clear framework. Rather than receiving ready-made solutions, we were encouraged to reflect, rethink, and justify our choices. While this level of freedom was sometimes challenging, especially during periods of uncertainty, it ultimately strengthened our sense of responsibility and ownership of the project.

We actively engaged with supervision by preparing questions, presenting our progress, and openly discussing uncertainties. This constructive engagement allowed supervision to function as a dialogue rather than a one-way evaluation. Feedback sessions often helped us confirm that we were on the right track or redirect our efforts when needed, particularly when dealing with complex elements such as class diagrams and system structure.

Reflecting forward, in future projects I would aim to engage even earlier and more proactively with supervision, especially when facing uncertainty. Asking focused questions sooner could reduce the need for later rework. At the same time, I would maintain the balance between independence and guidance, continuing to take responsibility for preparation and reflection rather than relying on supervisors for solutions.

7. Conclusion

Working on the Cloverville project provided us with valuable insight into what contributes to effective group work, especially in a development-oriented and academically demanding context. As a small group of three, we experienced both the advantages and challenges of close collaboration, shared responsibility, and time pressure. Reflecting on this experience allowed us to identify clear practices that supported our success, as well as areas that could be improved in future projects.

One of the most important factors for successful collaboration was clear communication and structure. Using tools such as Trello for task management and holding regular meetings helped us maintain an overview of progress and responsibilities. Daily check-ins during the implementation phase ensured that problems were identified early and addressed before they escalated. This proved especially important in a small group, where delays or misunderstandings had an immediate impact on the workload of others.

Another key lesson was the importance of addressing issues early and calmly. Minor conflicts arose when deadlines were missed or availability changed, but open discussion and mutual respect prevented these situations from developing into larger problems. This demonstrated our ability to manage complex situations constructively and adapt our collaboration when challenges appeared.

At the same time, the project highlighted what should be avoided in group work. Unclear expectations, delayed communication, or assuming that problems will resolve themselves can quickly create imbalance and stress, particularly in smaller teams. We learned that avoiding difficult conversations often leads to more work later, whereas early clarification saves time and energy.

Based on our experience, we created recommendations for successful group collaboration:

What to do:

- Define tasks, responsibilities, and deadlines clearly from the beginning.
- Use a shared planning tool to maintain transparency and accountability.
- Hold regular meetings to align progress and address challenges early.
- Communicate openly and respectfully about availability and workload.
- Support each other’s learning and share knowledge when challenges arise.

What not to do:

- Avoid unclear task ownership or assumptions about responsibilities.
- Do not postpone discussions about delays or misunderstandings.
- Avoid overloading individual group members without redistributing tasks.

Overall, the Cloverville project demonstrated that effective group work requires both structure and flexibility. By combining clear planning with open communication and mutual support, we were able to manage a complex development-oriented project successfully.

These experiences have strengthened our ability to collaborate professionally and will directly influence how we approach teamwork in future academic and professional contexts.

8. References

Cherry, K. (2025). The pros and cons of laissez-faire leadership (Theory Z). Verywell Mind. https://www.verywellmind.com/what-is-laissez-faire-leadership-2795316

Hancock, J. (2025). Theory X and Theory Y. MindTools. https://www.mindtools.com/adi3nc1/theory-x-and-theory-y/

Hoffman, R. (2023, September). Social loafing in psychology: Definition, examples & theory. Simply Psychology. https://www.simplypsychology.org/social-loafing.html

Jordan, T. (n.d.). Glasl's nine-stage model of conflict escalation [PDF]. University of Houston. https://www.law.uh.edu/blakely/advocacy-survey/Conflict%20Escalation%20Glasl.pdf

Kelley School of Business Executive Education. (2024, July 19). Conflict styles identified with the dual concerns model. Indiana University. https://blogs.iu.edu/keep/4-conflict-styles-identified-with-the-dual-concerns-model/

McLeod, S. (2025, October). Maslow's hierarchy of needs. Simply Psychology. https://www.simplypsychology.org/maslow.html

Niemiec, C. P., & Ryan, R. M. (2009). Autonomy, competence, and relatedness in the classroom: Applying self-determination theory to educational practice. Theory and Research in Education, 7(2), 133–144. https://doi.org/10.1177/1477878509104318

University of Copenhagen. (n.d.). ABC model: The ABCs of Mental Health https://psychology.ku.dk/abc/

MIT Human Resources. (n.d.). Tuckman's stages of team development. https://hr.mit.edu/learning-topics/teams/articles/stages-development

Jenkins, P. (2021). How I use Trello to organize my work and my life. Medium. https://parkermjenkins.medium.com/how-i-use-trello-to-organize-my-work-and-my-life-6c24e8b16d87

Cornell University. (2025). Problem-based learning. Cornell Center for Teaching Innovation. https://teaching.cornell.edu/teaching-resources/active-collaborative-learning/problem-based-learning